

```
// File: frontend/src/main.js
import { createApp } from 'vue'
import { createPinia } from 'pinia'
import ElementPlus from 'element-plus'
import 'element-plus/dist/index.css'
import 'highlight.js/styles/github.css'
import zhCn from 'element-plus/dist/locale/zh-cn.mjs'
import * as ElementPlusIconsVue from '@element-plus/icons-vue'

import App from './App.vue'
import router from './router'
import './assets/styles/main.scss'

const app = createApp(App)

// 注册 Element Plus 图标
for (const [key, component] of Object.entries(ElementPlusIconsVue)) {
  app.component(key, component)
}

app.use(createPinia())
app.use(router)
app.use(ElementPlus, { locale: zhCn })

app.mount('#app')

// File: frontend/src/router/index.js
import { createRouter, createWebHistory } from 'vue-router'
import { AUTH_ENABLED } from '@config/app'

const routes = [
  {
    path: '/login',
    name: 'Login',
    component: () => import('@views/Login.vue'),
    meta: { guest: true, title: '登录' },
  },
  {
    path: '/',
    component: () => import('@views/Layout.vue'),
    children: [
      { path: '', name: 'Home', component: () => import('@views/Dashboard.vue'), meta: { title: '首页' } },
      { path: 'research-data', name: 'ResearchData', component: () => import('@views/ResearchData.vue'), meta: { title: '研究数据' } },
      { path: 'parks', name: 'Parks', component: () => import('@views/Parks.vue'), meta: { title: '遗址公园' } },
      { path: 'parks/:id', name: 'ParkDetail', component: () => import('@views/ParkDetail.vue'), meta: { title: '公园详情' } },
    ],
  },
]
```

```

    { path: 'map', name: 'Map', component: () => import('@/views/MapView.vue'), meta: { title: '地图浏览',
flush: true, hideFooter: true } },
    { path: 'compare', name: 'Comparison', component: () => import('@/views/Comparison.vue'), meta:
{ title: '对比分析' } },
    { path: 'comparison', redirect: '/compare' },
    { path: 'knowledge-graph', name: 'KnowledgeGraph', component: () =>
import('@/views/KnowledgeGraph.vue'), meta: { title: '知识图谱', wide: true } },
    { path: 'assistant', name: 'Assistant', component: () => import('@/views/Chat.vue'), meta: { title:
'AI 助手', wide: true, hideFooter: true } },
    { path: 'chat', redirect: '/assistant' },
    { path: 'library', name: 'Library', component: () => import('@/views/KnowledgeBase.vue'), meta:
{ title: '知识库' } },
    { path: 'knowledge-base', redirect: '/library' },
    { path: 'bibliometrics', name: 'Bibliometrics', component: () => import('@/views/Bibliometrics.vue'),
meta: { title: '文献计量' } },
    { path: 'about', name: 'About', component: () => import('@/views/About.vue'), meta: { title: '关于我
们' } },
    { path: 'data-management', name: 'DataManagement', component: () =>
import('@/views/admin/DataManage.vue'), meta: { title: '数据管理', admin: true } },
    { path: 'admin', redirect: '/data-management' },
  ],
},
{ path: '/*', redirect: '/' },
]

```

```
const router = createRouter({ history: createWebHistory(), routes, scrollBehavior: () => ({ top: 0 }) })
```

```

router.beforeEach((to) => {
  document.title = `${to.meta.title || '研究平台'} - 遗址公园研究平台`
  if (!AUTH_ENABLED) return true
  if (to.meta.guest) return true
  if (to.meta.admin && !localStorage.getItem('token')) {
    return { path: '/login', query: { redirect: to.fullPath } }
  }
  return true
})

```

```
export default router
```

```
// File: backend/app/api/chat.py
```

```

import json
import uuid
from datetime import datetime, timezone
from fastapi import APIRouter, Depends
from sqlalchemy.orm import Session
from sse_starlette.sse import EventSourceResponse

```

```

from ..core.database import get_db, get_neo4j, get_chroma_collection
from ..core.auth import get_current_user
from ..models.user import User

```

```
from ..schemas.chat import ChatRequest, ChatResponse, ConversationMeta, ConversationListResponse
from ..services.rag_engine import RAGEngine

router = APIRouter()

# 存储会话（内存），包含元数据
conversation_store: dict = {}
MAX_CONVERSATIONS = 100

def _get_or_create_conversation(conversation_id: str, first_message: str = "") -> str:
    """获取或创建会话，返回 conversation_id"""
    if not conversation_id:
        conversation_id = str(uuid.uuid4())

    if conversation_id not in conversation_store:
        if len(conversation_store) >= MAX_CONVERSATIONS:
            oldest_key = next(iter(conversation_store))
            del conversation_store[oldest_key]
        conversation_store[conversation_id] = {
            "id": conversation_id,
            "title": first_message[:30] if first_message else "新对话",
            "created_at": datetime.now(timezone.utc).isoformat(),
            "last_active": datetime.now(timezone.utc).isoformat(),
            "messages": [],
        }
    else:
        conversation_store[conversation_id]["last_active"] = datetime.now(timezone.utc).isoformat()

    return conversation_id

@router.post("", response_model=ChatResponse)
async def chat(
    request: ChatRequest,
    db: Session = Depends(get_db),
    current_user: User = Depends(get_current_user),
):
    """聊天接口（非流式）"""
    conversation_id = _get_or_create_conversation(request.conversation_id, request.message)
    conv = conversation_store[conversation_id]
    history = conv["messages"]

    neo4j_session = get_neo4j()
    rag = RAGEngine(db=db, neo4j_session=neo4j_session, chroma=get_chroma_collection())

    result = await rag.query(
        question=request.message,
        conversation_history=history,
    )
```

```
history.append({"role": "user", "content": request.message})
history.append({"role": "assistant", "content": result["answer"]})

if len(history) > 20:
    conv["messages"] = history[-20:]

conv["title"] = conv["title"] or request.message[:30]
conv["last_active"] = datetime.now(timezone.utc).isoformat()

return ChatResponse(
    message=result["answer"],
    conversation_id=conversation_id,
    sources=result.get("sources", []),
    chart_data=result.get("chart_data"),
)

@router.post("/stream")
async def chat_stream(
    request: ChatRequest,
    db: Session = Depends(get_db),
    current_user: User = Depends(get_current_user),
):
    """流式聊天接口 (SSE)"""
    conversation_id = _get_or_create_conversation(request.conversation_id, request.message)
    conv = conversation_store[conversation_id]
    history = conv["messages"]

    neo4j_session = get_neo4j()
    rag = RAGEngine(db=db, neo4j_session=neo4j_session, chroma=get_chroma_collection())

    async def event_generator():
        full_answer = ""
        try:
            async for event in rag.stream_query(
                question=request.message,
                conversation_history=history,
            ):
                event_type = event["type"]
                event_data = event["data"]

                if event_type == "token":
                    full_answer += event_data
                    yield {"event": "token", "data": json.dumps({"token": event_data}, ensure_ascii=False)}
                elif event_type == "chart_data":
                    yield {"event": "chart_data", "data": json.dumps(event_data, ensure_ascii=False)}
                elif event_type == "sources":
                    yield {"event": "sources", "data": json.dumps(event_data, ensure_ascii=False)}
                elif event_type == "done":
```

```
        yield {"event": "done", "data": "{}"}

    # 保存对话历史
    history.append({"role": "user", "content": request.message})
    history.append({"role": "assistant", "content": full_answer})

    if len(history) > 20:
        conv["messages"] = history[-20:]

    conv["last_active"] = datetime.now(timezone.utc).isoformat()

except Exception as e:
    yield {"event": "error", "data": json.dumps({"detail": str(e)}, ensure_ascii=False)}

return EventSourceResponse(
    event_generator(),
    headers={
        "Cache-Control": "no-cache",
        "X-Accel-Buffering": "no",
        "Connection": "keep-alive",
    },
)

@router.get("/conversations", response_model=ConversationListResponse)
async def list_conversations(
    current_user: User = Depends(get_current_user),
):
    """获取所有会话列表"""
    convs = []
    for cid, cdata in conversation_store.items():
        convs.append(ConversationMeta(
            id=cdata["id"],
            title=cdata["title"],
            message_count=len(cdata["messages"]),
            created_at=cdata["created_at"],
            last_active=cdata["last_active"],
        ))
    convs.sort(key=lambda x: x.last_active, reverse=True)
    return ConversationListResponse(conversations=convs)

@router.get("/history/{conversation_id}")
async def get_history(
    conversation_id: str,
    current_user: User = Depends(get_current_user),
):
    """获取会话历史"""
    conv = conversation_store.get(conversation_id)
    if not conv:
```

```
        return []
    return conv["messages"]

@router.delete("/conversation/{conversation_id}")
async def delete_conversation(
    conversation_id: str,
    current_user: User = Depends(get_current_user),
):
    """删除会话"""
    if conversation_id in conversation_store:
        del conversation_store[conversation_id]
    return {"message": "对话已删除"}

@router.delete("/history/{conversation_id}")
async def clear_history(
    conversation_id: str,
    current_user: User = Depends(get_current_user),
):
    """清空会话历史（兼容旧接口）"""
    if conversation_id in conversation_store:
        del conversation_store[conversation_id]
    return {"message": "对话历史已清除"}

// File: backend/app/api/kg.py
from fastapi import APIRouter, Depends, Query
from sqlalchemy.orm import Session
from typing import Optional

from ..core.database import get_db, get_neo4j, neo4j_driver
from ..core.auth import get_current_user
from ..models.user import User
from ..models.park import Park
from ..models.score import Score
from ..models.indicator import Indicator

router = APIRouter()

def has_neo4j():
    """检查 Neo4j 是否可用"""
    return neo4j_driver is not None

@router.get("/stats")
async def kg_stats(
    db: Session = Depends(get_db),
    current_user: User = Depends(get_current_user),
):
```

```

"""知识图谱统计信息"""
if has_neo4j():
    # Neo4j 模式
    with get_neo4j() as session:
        node_count = session.run("MATCH (n) RETURN count(n) as c").single()["c"]
        rel_count = session.run("MATCH ()-[r]->() RETURN count(r) as c").single()["c"]
        return {
            "total_nodes": node_count,
            "total_relations": rel_count,
            "source": "neo4j",
        }
else:
    # SQLite 模式 - 从关系数据库生成统计
    parks_count = db.query(Park).count()
    indicators_count = db.query(Indicator).count()
    scores_count = db.query(Score).count()

    return {
        "total_nodes": parks_count + indicators_count,
        "total_relations": scores_count,
        "source": "sqlite",
        "node_types": [
            {"labels": ["Park"], "count": parks_count},
            {"labels": ["Indicator"], "count": indicators_count},
        ],
        "relation_types": [
            {"type": "HAS_SCORE", "count": scores_count},
        ],
    }

@router.get("/entity/{entity_type}")
async def get_entities(
    entity_type: str,
    name: Optional[str] = Query(None),
    limit: int = Query(50, ge=1, le=200),
    db: Session = Depends(get_db),
    current_user: User = Depends(get_current_user),
):
    """获取实体列表"""
    if has_neo4j():
        with get_neo4j() as session:
            if name:
                result = session.run(
                    f"MATCH (n:{entity_type}) WHERE n.name CONTAINS $name RETURN n LIMIT $limit",
                    name=name, limit=limit
                )
            else:
                result = session.run(f"MATCH (n:{entity_type}) RETURN n LIMIT $limit", limit=limit)
            return [dict(record["n"]) for record in result]

```

```

# SQLite 模式
if entity_type == "Park":
    query = db.query(Park)
    if name:
        query = query.filter(Park.name.contains(name) | Park.short_name.contains(name))
    parks = query.limit(limit).all()
    return [
        {
            "name": p.name,
            "short_name": p.short_name,
            "type": p.park_type,
            "province": p.province,
            "city": p.city,
            "batch": p.batch,
            "area": p.total_area,
        }
        for p in parks
    ]

elif entity_type == "Indicator":
    indicators = db.query(Indicator).limit(limit).all()
    return [
        {
            "code": i.code,
            "name": i.name,
            "dimension": i.dimension,
        }
        for i in indicators
    ]

return []

@router.get("/entity/{entity_type}/{name}")
async def get_entity_detail(
    entity_type: str,
    name: str,
    db: Session = Depends(get_db),
    current_user: User = Depends(get_current_user),
):
    """获取实体详情"""
    if has_neo4j():
        with get_neo4j() as session:
            result = session.run(
                f"MATCH (n:{entity_type} {{name: $name}}) OPTIONAL MATCH (n)-[r]->(m) RETURN n,
collect({{relation: type(r), target: m.name}}) as relations",
                name=name
            )
            record = result.single()

```



```

        if record:
            return {"entity": dict(record["n"]), "relations": record["relations"]}
        return {"error": "实体不存在"}

# SQLite 模式
if entity_type == "Park":
    park = db.query(Park).filter(
        (Park.name == name) | (Park.short_name == name)
    ).first()
    if park:
        scores = (
            db.query(Score, Indicator)
            .join(Indicator, Score.indicator_id == Indicator.id)
            .filter(Score.park_id == park.id)
            .all()
        )
        return {
            "entity": {
                "name": park.name,
                "short_name": park.short_name,
                "type": park.park_type,
                "province": park.province,
            },
            "relations": [
                {
                    "relation": "HAS_SCORE",
                    "target": f"{ind.code} {ind.name}",
                    "value": sc.normalized_score,
                }
                for sc, ind in scores
            ],
        }
    return {"error": "实体不存在"}

@router.get("/neighbors/{name}")
async def get_neighbors(
    name: str,
    depth: int = Query(2, ge=1, le=3),
    db: Session = Depends(get_db),
    current_user: User = Depends(get_current_user),
):
    """获取邻居节点"""
    if has_neo4j():
        with get_neo4j() as session:
            result = session.run(
                f"MATCH path = (n {{name: $name}})-[*1..{depth}]- (m) RETURN path LIMIT 100",
                name=name
            )

```

```

        nodes = {}
        edges = []
        for record in result:
            path = record["path"]
            for node in path.nodes:
                nid = node.element_id
                if nid not in nodes:
                    nodes[nid] = dict(node)
            for rel in path.relationships:
                edges.append({
                    "source": rel.start_node.get("name"),
                    "target": rel.end_node.get("name"),
                    "type": rel.type,
                })
        return {"nodes": list(nodes.values()), "edges": edges}

# SQLite 模式 - 返回公园及其评分关系
park = db.query(Park).filter(
    (Park.name == name) | (Park.short_name == name)
).first()

if not park:
    # 返回所有公园的概览
    parks = db.query(Park).all()
    nodes = [{"name": p.short_name or p.name, "type": p.park_type, "labels": ["Park"]} for p in parks]
    edges = []

    # 同类型公园之间建立关系
    type_groups = {}
    for p in parks:
        if p.park_type not in type_groups:
            type_groups[p.park_type] = []
        type_groups[p.park_type].append(p.short_name or p.name)

    for ptype, names in type_groups.items():
        for i in range(len(names)):
            for j in range(i + 1, len(names)):
                edges.append({"source": names[i], "target": names[j], "type": "SIMILAR_TYPE"})

    return {"nodes": nodes, "edges": edges}

# 返回特定公园的详情
scores = (
    db.query(Score, Indicator)
    .join(Indicator, Score.indicator_id == Indicator.id)
    .filter(Score.park_id == park.id)
    .all()
)

nodes = [{"name": park.short_name or park.name, "type": park.park_type, "labels": ["Park"]}]
```

```

edges = []

for sc, ind in scores:
    nodes.append({
        "name": f"{ind.code} {ind.name}",
        "type": "Indicator",
        "labels": ["Indicator"],
        "score": sc.normalized_score,
    })
    edges.append({
        "source": park.short_name or park.name,
        "target": f"{ind.code} {ind.name}",
        "type": "HAS_SCORE",
        "value": sc.normalized_score,
    })

return {"nodes": nodes, "edges": edges}

@router.get("/search")
async def search_kg(
    q: str = Query(...),
    db: Session = Depends(get_db),
    current_user: User = Depends(get_current_user),
):
    """搜索知识图谱"""
    results = []

    # 搜索公园
    parks = db.query(Park).filter(
        Park.name.contains(q) | Park.short_name.contains(q)
    ).all()
    for p in parks:
        results.append({"labels": ["Park"], "data": {"name": p.name, "short_name": p.short_name, "type":
p.park_type}})

    # 搜索指标
    indicators = db.query(Indicator).filter(
        Indicator.name.contains(q) | Indicator.code.contains(q)
    ).all()
    for ind in indicators:
        results.append({"labels": ["Indicator"], "data": {"code": ind.code, "name": ind.name, "dimension":
ind.dimension}})

    return results

// File: backend/app/api/knowledge.py
"""知识库 API - 含 ChromaDB 索引、语义搜索、GB/T 7714 导出、健康检查、综述生成"""
import os
import json

```

```
import asyncio
from typing import List
from fastapi import APIRouter, Depends, HTTPException, UploadFile, File, Query
from sqlalchemy.orm import Session

from ..core.database import get_db
from ..core.auth import get_current_user, get_admin_user
from ..models.user import User
from ..models.literature import Literature, Citation, LitRelation
from ..services.kb_processor import extract_metadata, discover_relations
from ..services.kb_indexer import index_literature, delete_index, rebuild_index, semantic_search
from ..services.kb_lint import lint_knowledge_base, generate_literature_review

router = APIRouter()

# 存储路径
KB_DIR = os.path.join(os.path.dirname(os.path.dirname(os.path.dirname(__file__))), "knowledge_base")

def _ensure_kb_dirs():
    os.makedirs(os.path.join(KB_DIR, "raw"), exist_ok=True)
    os.makedirs(os.path.join(KB_DIR, "wiki"), exist_ok=True)

def _get_rel_target_title(db, rel, lit_id: int) -> str:
    """获取关系目标文献的标题，安全处理不存在的情况"""
    target_id = rel.target_id if rel.source_id == lit_id else rel.source_id
    if not target_id:
        return "未知"
    target = db.query(Literature).filter(Literature.id == target_id).first()
    return target.title if target else "未知"

# ===== 上传文献 =====

@router.post("/upload")
async def upload_literature(
    file: UploadFile = File(...),
    db: Session = Depends(get_db),
    current_user: User = Depends(get_admin_user),
):
    """上传文献并自动提取元数据 + 向量索引"""
    _ensure_kb_dirs()

    ext = file.filename.lower().rsplit('.', 1)[-1] if '.' in file.filename else ''
    if ext not in ('pdf', 'docx', 'doc', 'txt', 'md'):
        raise HTTPException(400, "仅支持 .pdf / .docx / .doc / .txt / .md")

    content = await file.read()
    if len(content) > 100 * 1024 * 1024:
```

```
        raise HTTPException(400, "文件不能超过 100MB")

# 提取文本
from ..services.doc_parser import extract_text
text = extract_text(content, file.filename)
if not text or len(text.strip()) < 50:
    raise HTTPException(400, "文件内容为空或无法识别")

# 保存原文
safe_name = file.filename.replace(" ", "_")
raw_path = os.path.join(KB_DIR, "raw", safe_name)
with open(raw_path, "wb") as f:
    f.write(content)

# LLM 提取元数据（线程池避免阻塞）
loop = asyncio.get_running_loop()
try:
    metadata = await asyncio.wait_for(
        loop.run_in_executor(None, extract_metadata, text),
        timeout=180.0,
    )
except asyncio.TimeoutError:
    metadata = {
        "title": file.filename, "year": 2024, "doc_type": "其他",
        "authors": [], "journal": "", "keywords": [], "abstract": text[:500],
        "funding": "", "methodology": "", "core_argument": "",
        "references": [], "parks_mentioned": [], "concepts_mentioned": [],
    }

# 保存数据库
lit = Literature(
    title=metadata.get("title", file.filename),
    authors=json.dumps(metadata.get("authors", []), ensure_ascii=False),
    year=metadata.get("year", 2024),
    journal=metadata.get("journal", ""),
    keywords=json.dumps(metadata.get("keywords", []), ensure_ascii=False),
    abstract=metadata.get("abstract", ""),
    doc_type=metadata.get("doc_type", "其他"),
    funding=metadata.get("funding", ""),
    methodology=metadata.get("methodology", ""),
    core_argument=metadata.get("core_argument", ""),
    file_path=raw_path,
    wiki_path="",
    raw_metadata=json.dumps(metadata, ensure_ascii=False),
)
db.add(lit)
db.flush()

# 保存引文关系
refs = metadata.get("references", [])
```

```
for ref in refs:
    if isinstance(ref, dict):
        existing = db.query(Literature).filter(
            Literature.title == ref.get("title", "")
        ).first()
        citation = Citation(
            source_id=lit.id,
            target_title=ref.get("title", ""),
            target_author=ref.get("author", ""),
            target_year=ref.get("year"),
            target_id=existing.id if existing else None,
        )
        db.add(citation)

db.commit()
db.refresh(lit)

# 向量索引
kws = metadata.get("keywords", [])
index_literature(lit.id, lit.title, lit.abstract or "", kws)

# 异步发现与已有文献的关系
try:
    existing = db.query(Literature).filter(Literature.id != lit.id).all()
    existing_data = [
        {"id": e.id, "title": e.title, "year": e.year, "abstract": (e.abstract or "")[:200]}
        for e in existing[:20]
    ]
    new_data = {"id": lit.id, "title": lit.title, "year": lit.year, "abstract": lit.abstract or ""}
    relations = await asyncio.wait_for(
        loop.run_in_executor(None, discover_relations, new_data, existing_data),
        timeout=120.0,
    )

    for rel in relations:
        target_title = rel.get("target_title", "")
        if not target_title:
            continue
        target = db.query(Literature).filter(Literature.title == target_title).first()
        if target:
            db.add(LitRelation(
                source_id=lit.id,
                target_id=target.id,
                relation_type=rel.get("type", "shares_topic"),
                strength=rel.get("strength", 0.5),
                explanation=rel.get("explanation", ""),
            ))
    db.commit()
except (asyncio.TimeoutError, Exception) as e:
    print(f"[KB] 关系发现跳过: {str(e)[:80]}")
```

```
    return {
        "id": lit.id,
        "title": lit.title,
        "year": lit.year,
        "keywords": metadata.get("keywords", []),
        "abstract": (lit.abstract or "")[:200],
        "citations_found": len(refs),
    }

}

# ===== 文献列表 =====

@router.get("/list")
async def list_literature(
    page: int = Query(1, ge=1),
    page_size: int = Query(20, ge=1, le=100),
    keyword: str = Query(None),
    year_from: int = Query(None),
    year_to: int = Query(None),
    doc_type: str = Query(None),
    db: Session = Depends(get_db),
    current_user: User = Depends(get_current_user),
):
    """列出文献"""
    query = db.query(Literature)
    if keyword:
        like = f"%{keyword}%"
        query = query.filter(
            Literature.title.like(like) |
            Literature.abstract.like(like) |
            Literature.keywords.like(like)
        )
    if year_from:
        query = query.filter(Literature.year >= year_from)
    if year_to:
        query = query.filter(Literature.year <= year_to)
    if doc_type:
        query = query.filter(Literature.doc_type == doc_type)

    total = query.count()
    items = query.order_by(Literature.created_at.desc()).offset(
        (page - 1) * page_size
    ).limit(page_size).all()

    return {
        "total": total,
        "page": page,
        "items": [
            {

```

```

        "id": l.id,
        "title": l.title,
        "authors": json.loads(l.authors) if l.authors else [],
        "year": l.year,
        "journal": l.journal,
        "keywords": json.loads(l.keywords) if l.keywords else [],
        "abstract": (l.abstract or "")[:300],
        "doc_type": l.doc_type,
        "citation_count": l.citation_count,
        "created_at": str(l.created_at),
    }
    for l in items
],
}

# ===== 文献详情 =====

@router.get("/detail/{lit_id}")
async def get_literature_detail(
    lit_id: int,
    db: Session = Depends(get_db),
    current_user: User = Depends(get_current_user),
):
    """文献详情 + 关联文献"""
    lit = db.query(Literature).filter(Literature.id == lit_id).first()
    if not lit:
        raise HTTPException(404, "文献不存在")

    citations_from = db.query(Citation).filter(Citation.source_id == lit_id).all()
    citations_to = db.query(Citation).filter(Citation.target_id == lit_id).all()

    relations = db.query(LitRelation).filter(
        (LitRelation.source_id == lit_id) | (LitRelation.target_id == lit_id)
    ).all()

    kws = json.loads(lit.keywords) if lit.keywords else []
    related = []
    if kws:
        conditions = [Literature.keywords.like(f"%{kw}%") for kw in kws[:5]]
        from sqlalchemy import or_
        related = db.query(Literature).filter(
            Literature.id != lit_id,
            or_(*conditions),
        ).limit(10).all()

    return {
        "id": lit.id,
        "title": lit.title,
        "authors": json.loads(lit.authors) if lit.authors else [],

```



```

        "year": lit.year,
        "journal": lit.journal,
        "keywords": kws,
        "abstract": lit.abstract,
        "doc_type": lit.doc_type,
        "funding": lit.funding,
        "methodology": lit.methodology,
        "core_argument": lit.core_argument,
        "citation_count": lit.citation_count,
        "created_at": str(lit.created_at),
        "citations_from": [
            {"title": c.target_title, "author": c.target_author, "year": c.target_year, "in_db":
bool(c.target_id)}
            for c in citations_from
        ],
        "cited_by": [
            {"title": c.source.title, "year": c.source.year} if c.source else {"title": "未知"}
            for c in citations_to
        ],
        "relations": [
            {
                "target_title": _get_rel_target_title(db, r, lit_id),
                "type": r.relation_type,
                "explanation": r.explanation,
            }
            for r in relations
        ],
        "related": [
            {"id": r.id, "title": r.title, "year": r.year}
            for r in related
        ],
    }
}

```

===== 删除文献 =====

```

@router.delete("/delete/{lit_id}")
async def delete_literature(
    lit_id: int,
    db: Session = Depends(get_db),
    current_user: User = Depends(get_admin_user),
):
    """删除文献"""
    lit = db.query(Literature).filter(Literature.id == lit_id).first()
    if not lit:
        raise HTTPException(404, "文献不存在")
    if lit.file_path and os.path.exists(lit.file_path):
        os.remove(lit.file_path)
    db.delete(lit)
    db.commit()

```

```
# 删除向量索引
delete_index(lit_id)
return {"message": "已删除"}

# ===== 文献计量 =====

@router.get("/bibliometrics")
async def get_bibliometrics(
    db: Session = Depends(get_db),
    current_user: User = Depends(get_current_user),
):
    """获取全库文献计量指标"""
    from ..services.bibliometrics import compute_all
    loop = asyncio.get_running_loop()
    return await loop.run_in_executor(None, compute_all, db)

# ===== 语义搜索（ChromaDB 优先，SQL 兜底） =====

@router.get("/search")
async def search_literature(
    q: str = Query(...),
    limit: int = Query(10, ge=1, le=50),
    db: Session = Depends(get_db),
    current_user: User = Depends(get_current_user),
):
    """语义搜索（ChromaDB 向量检索优先，SQL LIKE 兜底）"""
    results = semantic_search(q, limit, db)
    return {"query": q, "results": results, "mode": "semantic" if any(r.get("similarity") for r in results)
    else "sql"}

# ===== 向量索引管理 =====

@router.post("/reindex")
async def reindex_knowledge_base(
    db: Session = Depends(get_db),
    current_user: User = Depends(get_admin_user),
):
    """全量重建文献向量索引"""
    loop = asyncio.get_running_loop()
    result = await loop.run_in_executor(None, rebuild_index, db)
    return result

# ===== GB/T 7714 引用格式导出 =====

@router.get("/cite/{lit_id}")
async def get_citation_gbt7714(
```

```

        lit_id: int,
        style: str = Query("gbt7714", description="引用格式: gbt7714 | apa | mla"),
        db: Session = Depends(get_db),
        current_user: User = Depends(get_current_user),
    ):
        """获取 GB/T 7714 / APA / MLA 格式引用"""
        lit = db.query(Literature).filter(Literature.id == lit_id).first()
        if not lit:
            raise HTTPException(404, "文献不存在")

        authors = json.loads(lit.authors) if lit.authors else []
        formatted = _format_citation(lit, authors, style)
        return {"lit_id": lit_id, "style": style, "citation": formatted}

@router.post("/cite/batch")
async def batch_citation(
    lit_ids: List[int],
    style: str = Query("gbt7714"),
    db: Session = Depends(get_db),
    current_user: User = Depends(get_current_user),
):
    """批量生成引用"""
    lits = db.query(Literature).filter(Literature.id.in_(lit_ids)).all()
    id_map = {l.id: l for l in lits}
    results = []
    for lid in lit_ids:
        lit = id_map.get(lid)
        if lit:
            authors = json.loads(lit.authors) if lit.authors else []
            results.append({
                "lit_id": lid,
                "title": lit.title,
                "citation": _format_citation(lit, authors, style),
            })
    return {"style": style, "citations": results}

def _format_citation(lit, authors: list, style: str) -> str:
    """格式化引用"""
    author_str = ""
    if authors:
        names = [a.get("name", "") for a in authors[:3] if isinstance(a, dict)]
        if names:
            author_str = ", ".join(names)
            if len(authors) > 3:
                author_str += ", 等" if style == "gbt7714" else " et al."

    year = str(lit.year) if lit.year else ""
    title = lit.title or ""

```

```

if style == "gbt7714":
    # GB/T 7714-2015
    parts = []
    if author_str:
        parts.append(author_str + ".")
    parts.append(f"{title}[{lit.doc_type or 'J'}].")
    if lit.journal:
        parts.append(f"{lit.journal}.")
    if year:
        parts.append(f"{year}.")
    return " ".join(parts)

elif style == "apa":
    parts = []
    if author_str:
        parts.append(f"{author_str} ({year}).")
    else:
        parts.append(f"({year}).")
    parts.append(f"{title}.")
    if lit.journal:
        parts.append(f"*{lit.journal}*.")
    return " ".join(parts)

elif style == "mla":
    parts = []
    if author_str:
        parts.append(f'{author_str}. "{title}."')
    else:
        parts.append(f'"{title}."')
    if lit.journal:
        parts.append(f"*{lit.journal}*")
    if year:
        parts.append(f"{year}.")
    return " ".join(parts)

return f"{author_str}. {title}. {year}."

# ===== 知识库图谱数据 =====

@router.get("/graph")
async def get_knowledge_graph(
    db: Session = Depends(get_db),
    current_user: User = Depends(get_current_user),
):
    """获取文献知识图谱数据（引用网络 + 语义关系 + 关键词关联）"""
    lits = db.query(Literature).all()
    citations = db.query(Citation).all()
    relations = db.query(LitRelation).all()

```

```

nodes = []
edges = []
lit_ids = {l.id for l in lits}

# 文献节点
for l in lits:
    kws = json.loads(l.keywords) if l.keywords else []
    authors = json.loads(l.authors) if l.authors else []
    author_names = [a.get("name", "") for a in authors[:2] if isinstance(a, dict)]
    nodes.append({
        "id": f"lit_{l.id}",
        "label": l.title[:30],
        "type": "literature",
        "year": l.year,
        "doc_type": l.doc_type,
        "keywords": kws[:5],
        "authors": author_names,
        "symbolSize": 20 + len(kws) * 2,
    })

# 关键词节点（高频）
kw_counter = {}
for l in lits:
    kws = json.loads(l.keywords) if l.keywords else []
    for kw in kws:
        if kw not in kw_counter:
            kw_counter[kw] = {"count": 0, "lits": []}
        kw_counter[kw]["count"] += 1
        kw_counter[kw]["lits"].append(l.id)

top_kws = sorted(kw_counter.items(), key=lambda x: x[1]["count"], reverse=True)[:30]
for kw, info in top_kws:
    nodes.append({
        "id": f"kw_{kw}",
        "label": kw,
        "type": "keyword",
        "count": info["count"],
        "symbolSize": 10 + info["count"] * 3,
    })

# 关键词 → 文献边
for lit_id in info["lits"]:
    edges.append({
        "source": f"kw_{kw}",
        "target": f"lit_{lit_id}",
        "type": "has_keyword",
    })

# 引用边
for c in citations:

```

```

        if c.source_id in lit_ids:
            edges.append({
                "source": f"lit_{c.source_id}",
                "target": f"lit_{c.target_id}" if c.target_id and c.target_id in lit_ids else
f"lit_{c.source_id}",
                "type": "cites",
                "label": f"[{c.target_year or '?'}] {c.target_title[:20]}",
            })

# 语义关系边
for r in relations:
    if r.source_id in lit_ids and r.target_id in lit_ids:
        edges.append({
            "source": f"lit_{r.source_id}",
            "target": f"lit_{r.target_id}",
            "type": r.relation_type,
            "label": r.relation_type,
        })

    return {"nodes": nodes, "edges": edges, "total": len(nodes)}

# ===== 知识库健康检查 =====

@router.get("/lint")
async def lint_kb(
    db: Session = Depends(get_db),
    current_user: User = Depends(get_admin_user),
):
    """知识库健康检查"""
    loop = asyncio.get_running_loop()
    return await loop.run_in_executor(None, lint_knowledge_base, db)

# ===== 自动综述 =====

@router.get("/review")
async def get_literature_review(
    topic: str = Query("", description="综述主题，为空则全库综述"),
    max_lits: int = Query(20, ge=5, le=50),
    db: Session = Depends(get_db),
    current_user: User = Depends(get_current_user),
):
    """生成文献综述结构化数据"""
    loop = asyncio.get_running_loop()
    result = await loop.run_in_executor(None, generate_literature_review, db, topic, max_lits)
    return result

@router.post("/review/generate")

```

```
async def generate_review_with_llm(
    topic: str = Query("", description="综述主题"),
    max_lits: int = Query(20, ge=5, le=50),
    db: Session = Depends(get_db),
    current_user: User = Depends(get_admin_user),
):
    """使用 LLM 生成完整文献综述"""
    from ..core.config import settings

    if not settings.ai_available:
        raise HTTPException(400, "需要配置 AI API key 才能生成综述")

    # 先获取结构化数据
    data = generate_literature_review(db, topic, max_lits)
    if "error" in data:
        return data

    # 拼接 LLM 提示词
    lit_texts = []
    for lit in data["lits"]:
        authors = ", ".join([a.get("name", "") for a in lit["authors"][:3]])
        lit_texts.append(
            f"- [{lit['year']}] {authors}. {lit['title']}. "
            f"论点: {lit.get('core_argument', '')}. 方法: {lit.get('methodology', '')}"
        )

    top_kws = ", ".join(data.get("top_keywords", [])[:10])
    methods = ", ".join([m["name"] for m in data.get("methods", [])[:5]])

    prompt = f"""你是文化遗产保护研究领域的资深学者。请根据以下文献数据，撰写一篇学术综述。

## 综述主题
{topic or "文化遗产保护研究"}

## 文献概况
- 相关文献: {data['total_found']} 篇
- 时间跨度: {data.get('years_range', [])}
- 高频关键词: {top_kws}
- 常用方法: {methods}

## 文献列表
{chr(10).join(lit_texts)}

## 要求
1. 按研究脉络分段: 研究缘起 → 主要议题 → 方法论演进 → 前沿趋势 → 研究展望
2. 使用学术语言, 标注引用 (如 [1]、[2])
3. 指出研究空白和方法论局限
4. 1500-2500 字
5. 使用 Markdown 格式"""
```

```

try:
    from openai import OpenAI
    client = OpenAI(
        api_key=settings.DASHSCOPE_API_KEY,
        base_url=settings.LLM_BASE_URL or "https://dashscope.aliyuncs.com/compatible-mode/v1",
        timeout=120.0,
    )
    response = client.chat.completions.create(
        model=settings.AI_MODEL,
        messages=[
            {"role": "system", "content": "你是文化遗产保护研究领域的资深学者，擅长撰写文献综述。"},
            {"role": "user", "content": prompt},
        ],
        max_tokens=4096,
        temperature=0.7,
    )
    review_text = response.choices[0].message.content

    return {
        "topic": topic or "文化遗产保护",
        "lits_referenced": data["total_found"],
        "review": review_text,
        "references": [
            {"id": lit["id"], "title": lit["title"], "year": lit["year"]}
            for lit in data["lits"]
        ],
    }
except Exception as e:
    raise HTTPException(500, f"综述生成失败: {str(e)}")

// File: backend/app/api/parks.py
from fastapi import APIRouter, Depends, HTTPException, Query
from sqlalchemy.orm import Session
from typing import Optional, List

from ..core.database import get_db
from ..core.auth import get_current_user
from ..models.park import Park
from ..models.score import Score
from ..models.indicator import Indicator
from ..schemas.park import ParkResponse, ParkListResponse, ScoreInfo

router = APIRouter()

def build_park_response(park, db):
    """构建公园响应数据"""
    scores = (
        db.query(Score, Indicator)
        .join(Indicator, Score.indicator_id == Indicator.id)

```



```
        .filter(Score.park_id == park.id)
        .all()
    )
    score_list = [
        ScoreInfo(
            indicator_code=ind.code,
            indicator_name=ind.name,
            dimension=ind.dimension,
            score=sc.normalized_score,
            grade=sc.grade,
        )
        for sc, ind in scores
    ]

    return ParkResponse(
        id=park.id,
        name=park.name,
        short_name=park.short_name,
        park_type=park.park_type,
        batch=park.batch,
        province=park.province,
        city=park.city,
        district=park.district,
        longitude=park.longitude,
        latitude=park.latitude,
        total_area=park.total_area,
        core_area=park.core_area,
        world_heritage=park.world_heritage or 0,
        aaa_level=park.aaa_level,
        open_year=park.open_year,
        description=park.description,
        scores=score_list,
    )

@router.get("", response_model=ParkListResponse)
async def list_parks(
    park_type: Optional[str] = Query(None),
    province: Optional[str] = Query(None),
    batch: Optional[int] = Query(None),
    keyword: Optional[str] = Query(None),
    page: int = Query(1, ge=1),
    page_size: int = Query(20, ge=1, le=100),
    db: Session = Depends(get_db),
):
    query = db.query(Park)

    if park_type:
        query = query.filter(Park.park_type == park_type)
    if province:
```

```
        query = query.filter(Park.province == province)
    if batch:
        query = query.filter(Park.batch == batch)
    if keyword:
        query = query.filter(
            Park.name.contains(keyword) | Park.short_name.contains(keyword)
        )

    total = query.count()
    parks = query.offset((page - 1) * page_size).limit(page_size).all()

    result = [build_park_response(park, db) for park in parks]

    return ParkListResponse(total=total, items=result)

@router.get("/{park_id}", response_model=ParkResponse)
async def get_park(park_id: int, db: Session = Depends(get_db)):
    park = db.query(Park).filter(Park.id == park_id).first()
    if not park:
        raise HTTPException(status_code=404, detail="遗址公园不存在")
    return build_park_response(park, db)

@router.get("/{park_id}/sites")
async def get_park_sites(park_id: int, db: Session = Depends(get_db)):
    from ..models.site import Site
    park = db.query(Park).filter(Park.id == park_id).first()
    if not park:
        raise HTTPException(status_code=404, detail="遗址公园不存在")
    sites = db.query(Site).filter(Site.park_id == park_id).all()
    return [
        {
            "id": s.id, "site_name": s.site_name, "site_type": s.site_type,
            "period": s.period, "integrity_score": s.integrity_score,
            "safety_score": s.safety_score, "description": s.description,
        }
        for s in sites
    ]

@router.get("/{park_id}/scores")
async def get_park_scores(park_id: int, db: Session = Depends(get_db)):
    park = db.query(Park).filter(Park.id == park_id).first()
    if not park:
        raise HTTPException(status_code=404, detail="遗址公园不存在")

    scores = (
        db.query(Score, Indicator)
        .join(Indicator, Score.indicator_id == Indicator.id)
```

```
        .filter(Score.park_id == park_id)
        .order_by(Indicator.code)
        .all()
    )
    return [
        {
            "code": ind.code,
            "name": ind.name,
            "dimension": ind.dimension,
            "sub_dimension": ind.sub_dimension,
            "score": sc.normalized_score,
            "grade": sc.grade,
            "evidence": sc.evidence,
        }
        for sc, ind in scores
    ]

// File: backend/app/api/statistics.py
from fastapi import APIRouter, Depends, Query
from sqlalchemy.orm import Session
from sqlalchemy import func

from ..core.database import get_db
from ..core.auth import get_current_user
from ..models.park import Park
from ..models.site import Site
from ..models.score import Score
from ..models.indicator import Indicator
from ..models.user import User

router = APIRouter()

@router.get("/overview")
async def overview(
    db: Session = Depends(get_db),
    current_user: User = Depends(get_current_user),
):
    """数据概览"""
    total_parks = db.query(Park).count()
    total_sites = db.query(Site).count()
    total_indicators = db.query(Indicator).count()

    # 按类型统计
    type_stats = (
        db.query(Park.park_type, func.count(Park.id))
        .group_by(Park.park_type)
        .all()
    )
```

```
# 按省份统计
province_stats = (
    db.query(Park.province, func.count(Park.id))
    .group_by(Park.province)
    .all()
)

# 按批次统计
batch_stats = (
    db.query(Park.batch, func.count(Park.id))
    .filter(Park.batch.isnot(None))
    .group_by(Park.batch)
    .all()
)

return {
    "total_parks": total_parks,
    "total_sites": total_sites,
    "total_indicators": total_indicators,
    "by_type": {t: c for t, c in type_stats},
    "by_province": {p: c for p, c in province_stats},
    "by_batch": {b: c for b, c in batch_stats},
}

@router.get("/comparison")
async def comparison(
    park_ids: str = Query(..., description="公园 ID 列表, 逗号分隔"),
    dimension: str = Query(None, description="评估维度"),
    db: Session = Depends(get_db),
    current_user: User = Depends(get_current_user),
):
    """多公园对比"""
    ids = [int(x.strip()) for x in park_ids.split(",")]
    if len(ids) > 10:
        ids = ids[:10]

    parks = db.query(Park).filter(Park.id.in_(ids)).all()

    result = []
    for park in parks:
        query = (
            db.query(Score, Indicator)
            .join(Indicator, Score.indicator_id == Indicator.id)
            .filter(Score.park_id == park.id)
        )
        if dimension:
            query = query.filter(Indicator.dimension == dimension)

        scores = query.all()
```

```
        score_data = {}
        for sc, ind in scores:
            score_data[ind.code] = {
                "name": ind.name,
                "dimension": ind.dimension,
                "score": sc.normalized_score,
                "grade": sc.grade,
            }

        result.append({
            "park_id": park.id,
            "park_name": park.short_name or park.name,
            "park_type": park.park_type,
            "scores": score_data,
        })

    return result


@router.get("/dimension")
async def dimension_scores(
    park_type: str = Query(None, description="公园类型"),
    db: Session = Depends(get_db),
    current_user: User = Depends(get_current_user),
):
    """各维度得分统计"""
    query = (
        db.query(
            Park.park_type,
            Indicator.dimension,
            func.avg(Score.normalized_score).label("avg_score"),
            func.count(Score.id).label("count"),
        )
        .join(Score, Park.id == Score.park_id)
        .join(Indicator, Score.indicator_id == Indicator.id)
    )

    if park_type:
        query = query.filter(Park.park_type == park_type)

    results = query.group_by(Park.park_type, Indicator.dimension).all()

    return [
        {
            "park_type": r.park_type,
            "dimension": r.dimension,
            "avg_score": round(float(r.avg_score), 2),
            "count": r.count,
        }
    ]
```

```

        for r in results
    ]

@router.get("/radar/{park_id}")
async def radar_data(
    park_id: int,
    db: Session = Depends(get_db),
    current_user: User = Depends(get_current_user),
):
    """获取单个公园的雷达图数据"""
    park = db.query(Park).filter(Park.id == park_id).first()
    if not park:
        return {"error": "公园不存在"}

    scores = (
        db.query(Indicator.dimension, func.avg(Score.normalized_score).label("avg"))
        .join(Score, Indicator.id == Score.indicator_id)
        .filter(Score.park_id == park_id)
        .group_by(Indicator.dimension)
        .all()
    )

    return {
        "park_name": park.short_name or park.name,
        "dimensions": [
            {"dimension": r.dimension, "score": round(float(r.avg), 2)}
            for r in scores
        ],
    }

// File: backend/app/services/rag_engine.py
"""
RAG 问答引擎
- 保留本地模板模式作为降级方案
- 当配置了真实 AI key 时, 调用 LLM API 生成回答
- 支持流式 (SSE) 输出
- 集成知识库文献检索
"""

import json
from typing import List, Dict, Any, Optional, AsyncGenerator

from ..core.config import settings
from .llm_provider import get_llm_provider, MockProvider
from .prompt_builder import build_messages

class RAGEngine:
    """基于知识图谱的 RAG 问答引擎"""

```

```

def __init__(self, db, neo4j_session=None, chroma=None):
    self.db = db
    self.neo4j = neo4j_session
    self.chroma = chroma
    self.llm = get_llm_provider()

async def query(
    self,
    question: str,
    conversation_history: List[Dict] = None,
) -> Dict[str, Any]:
    """处理用户查询"""
    # 1-4: RAG 管线（与本地模式共用）
    intent = self._classify_intent(question)
    db_context = self._query_database(question, intent)
    doc_context = self._query_vector_store(question) if self.chroma else ""
    lit_context = self._query_knowledge_base(question)
    context = self._assemble_context(db_context, doc_context, lit_context)
    chart_data = self._generate_chart_data(question, intent)
    if results and results["documents"]:
        return "\n".join([f"相关文档: {doc[:300]}" for doc in results["documents"][0]])
    except Exception:
        pass
    return ""

def _query_knowledge_base(self, question: str) -> str:
    """
    查询文献知识库
    判断用户问题是否涉及学术/遗产保护理论，若是则检索文献
    同时拉取每篇文献的引用和关联关系
    """

```

```
// ===== 加载数据 =====

async function loadParks() {
  const res = await api.get('/admin/parks')
  parks.value = res.data
}

async function loadSites() {
  if (!siteFilterParkId.value) {
    sites.value = []
    return
  }
  const res = await api.get(`/admin/parks/${siteFilterParkId.value}/sites`)
  sites.value = res.data
}

async function loadScores() {
  const params = scoreFilterParkId.value ? { park_id: scoreFilterParkId.value } : {}
  const res = await api.get('/admin/scores', { params })
  scores.value = res.data
}

async function loadIndicators() {
  try {
    const res = await api.get('/admin/indicators')
    indicators.value = res.data
  } catch (e) {
    console.error('加载指标列表失败', e)
  }
}

async function loadUsers() {
  const res = await api.get('/admin/users')
  users.value = res.data
}

// ===== 公园操作 =====

function showParkDialog(row = null) {
  parkForm.value = row ? { ...row } : {
    name: '', short_name: '', park_type: '', batch: 1,
    province: '', city: '', longitude: null, latitude: null,
    total_area: null, aaa_level: '', description: ''
  }
  parkDialogVisible.value = true
}

async function savePark() {
  try {
    if (parkForm.value.id) {
```



```
        await api.put(`/admin/parks/${parkForm.value.id}`, parkForm.value)
      } else {
        await api.post('/admin/parks', parkForm.value)
      }
      ElMessage.success('保存成功')
      parkDialogVisible.value = false
      loadParks()
    } catch (e) {
      ElMessage.error(e.response?.data?.detail || '保存失败')
    }
  }

async function deletePark(id) {
  await ElMessageBox.confirm('确定删除此公园？关联数据将一并删除。', '确认')
  await api.delete(`/admin/parks/${id}`)
  ElMessage.success('删除成功')
  loadParks()
}

// ===== 遗址点操作 =====

function showSiteDialog(_row = null) {
  // 简化处理
  ElMessage.info('请使用 Excel 导入遗址点数据')
}

async function deleteSite(id) {
  await ElMessageBox.confirm('确定删除？', '确认')
  await api.delete(`/admin/sites/${id}`)
  ElMessage.success('删除成功')
  loadSites()
}

// ===== 评分操作 =====

function showScoreDialog(row = null) {
  scoreForm.value = row ? { ...row } : {
    park_id: null, indicator_id: null,
    normalized_score: 60, evidence: '', data_year: 2025
  }
  scoreDialogVisible.value = true
}

async function saveScore() {
  try {
    if (scoreForm.value.id) {
      await api.put(`/admin/scores/${scoreForm.value.id}`, scoreForm.value)
    } else {
      await api.post('/admin/scores', scoreForm.value)
    }
  }
```

```
      ElMessage.success('保存成功')
      scoreDialogVisible.value = false
      loadScores()
    } catch (e) {
      ElMessage.error(e.response?.data?.detail || '保存失败')
    }
  }

  async function deleteScore(id) {
    await ElMessageBox.confirm('确定删除?', '确认')
    await api.delete(`/admin/scores/${id}`)
    ElMessage.success('删除成功')
    loadScores()
  }

  // ===== 用户操作 =====

  async function updateRole(userId, role) {
    await api.put(`/admin/users/${userId}/role`, null, { params: { role } })
    ElMessage.success('角色已更新')
  }

  async function toggleStatus(userId) {
    await api.put(`/admin/users/${userId}/status`)
    ElMessage.success('状态已更新')
    loadUsers()
  }

  // ===== 导入回调 =====

  function handleImportSuccess(res) {
    if (res.success) {
      ElMessage.success(`导入成功: ${res.success}条`)
      loadParks()
      loadScores()
      loadSites()
    }
    if (res.errors?.length) {
      ElMessage.warning(`${res.errors.length}条导入失败`)
    }
  }
</script>

<style scoped lang="scss">
.admin-tabs-card {
  padding: 0 22px 22px;
  overflow: hidden;
}

.admin-tabs-card :deep(.el-tabs__header) {
```

```
margin-bottom: 20px;
}

.admin-tabs-card :deep(.el-tabs__nav-wrap) {
padding: 0 6px;
}

.toolbar {
display: flex;
align-items: center;
flex-wrap: wrap;
gap: 12px;
margin-bottom: 16px;
}

.toolbar a { text-decoration: none; }

@media (max-width: 760px) {
.admin-tabs-card { padding-inline: 12px; }
.toolbar { align-items: stretch; }
.toolbar .el-select { width: 100%; }
}
</style>

// File: frontend/src/views/admin/ReportImport.vue
<template>
<div class="report-import">
<!-- 上传区 -->
<div v-if="!parsed" class="upload-section">
<el-upload
ref="uploadRef"
class="upload-box"
drag
:auto-upload="false"
:limit="1"
accept=".docx,.doc,.txt"
:on-change="handleFileChange"
:on-exceed="() => ElMessage.warning('只能上传一个文件')"
>
<el-icon class="upload-icon"><UploadFilled /></el-icon>
<div class="upload-text">
<p><strong>拖拽或点击上传调研报告</strong></p>
<p class="upload-hint">支持 .docx / .doc 格式, 最大 50MB</p>
</div>
</el-upload>

<div v-if="uploadFile" class="upload-info">
<el-tag type="info" size="large">{{ uploadFile.name }}</el-tag>
<el-button
type="primary"

```

```

      :loading="parsing"
      @click="startParse"
      style="margin-left:12px"
    >
      {{ parsing ? 'AI 解析中...' : '开始解析' }}
    </el-button>
  </div>

  <div v-if="parsing" class="parse-progress">
    <el-progress :percentage="parseProgress" :stroke-width="16" :text-inside="true" />
    <p class="progress-text">{{ parseStatus }}</p>
  </div>
</div>

<!-- 解析结果预览 -->
<div v-if="parsed" class="result-section">
  <div class="result-header">
    <h3>解析结果预览</h3>
    <div>
      <el-tag type="success">{{ parsed.parks?.length || 0 }} 个公园</el-tag>
      <el-tag type="warning" style="margin-left:8px">{{ parsed.scores?.length || 0 }} 条评分</el-tag>
      <el-tag type="info" style="margin-left:8px">{{ parsed.surveys?.length || 0 }} 条问卷</el-tag>
    </div>
  </div>
</div>

<!-- 公园列表 -->
<el-card header="遗址公园" shadow="never" style="margin-top:16px">
  <el-table :data="parsed.parks" stripe border max-height="400">
    <el-table-column prop="name" label="名称" min-width="200" />
    <el-table-column prop="short_name" label="简称" width="120" />
    <el-table-column prop="park_type" label="类型" width="80" />
    <el-table-column prop="province" label="省份" width="80" />
    <el-table-column prop="city" label="城市" width="100" />
    <el-table-column label="操作" width="80">
      <template #default="{ $index }">
        <el-button size="small" type="danger" text @click="removePark($index)">移除</el-button>
      </template>
    </el-table-column>
  </el-table>
  <el-empty v-if="!parsed.parks?.length" description="未识别到公园数据" />
</el-card>

<!-- 评分列表 -->
<el-card header="评估评分" shadow="never" style="margin-top:16px">
  <el-table :data="parsed.scores" stripe border max-height="400">
    <el-table-column prop="park_name" label="公园" width="120" />
    <el-table-column prop="indicator_code" label="编号" width="80" />
    <el-table-column prop="indicator_name" label="指标" min-width="160" />
    <el-table-column prop="score" label="得分" width="80">
      <template #default="{ row }">

```

```

      <el-tag :type="row.score >= 80 ? 'success' : row.score >= 60 ? 'primary' : row.score >= 40 ?
'warning' : 'danger'">
        {{ row.score }}
      </el-tag>
    </template>
  </el-table-column>
  <el-table-column prop="dimension" label="维度" width="140" />
  <el-table-column label="操作" width="80">
    <template #default="{ $index }">
      <el-button size="small" type="danger" text @click="removeScore($index)">移除</el-button>
    </template>
  </el-table-column>
</el-table>
<el-empty v-if="!parsed.scores?.length" description="未识别到评分数据" />
</el-card>

<!-- 问卷列表 -->
<el-card v-if="parsed.surveys?.length" header="调研问卷" shadow="never" style="margin-top:16px">
  <el-table :data="parsed.surveys" stripe border>
    <el-table-column prop="park_name" label="公园" width="120" />
    <el-table-column prop="total_distributed" label="发放" width="80" />
    <el-table-column prop="total_collected" label="回收" width="80" />
    <el-table-column prop="valid_count" label="有效" width="80" />
    <el-table-column prop="sampling_method" label="抽样方法" />
  </el-table>
</el-card>

<!-- 操作按钮 -->
<div class="result-actions">
  <el-button @click="reset">重新上传</el-button>
  <el-button type="primary" :loading="importing" @click="confirmImport">
    {{ importing ? '导入中...' : '确认导入' }}
  </el-button>
</div>
</div>
</div>
</template>

<script setup>
import { ref } from 'vue'
import { ElMessage, ElMessageBox } from 'element-plus'
import api from '@/utils/api'

const uploadRef = ref(null)
const uploadFile = ref(null)
const parsing = ref(false)
const importing = ref(false)
const parsed = ref(null)
const parseProgress = ref(0)
const parseStatus = ref('')

```

```
function handleFileChange(file) {
  uploadFile.value = file.raw
}

async function startParse() {
  if (!uploadFile.value) return

  parsing.value = true
  parseProgress.value = 30
  parseStatus.value = '正在提取文档文本...'

  try {
    const formData = new FormData()
    formData.append('file', uploadFile.value)

    parseProgress.value = 60
    parseStatus.value = 'AI 正在解析文档内容...'

    const res = await api.post('/admin/upload/report', formData, {
      headers: { 'Content-Type': 'multipart/form-data' },
      timeout: 180000,
    })

    parseProgress.value = 100
    parseStatus.value = '解析完成!'
    parsed.value = res.data.parsed

    setTimeout(() => {
      if (parsed.value) {
        ElMessage.success(
          `解析完成: ${res.data.parsed.parks.length} 个公园, ${res.data.parsed.scores.length} 条评分`
        )
      }
    }, 300)
  } catch (e) {
    ElMessage.error(e.response?.data?.detail || '解析失败')
  } finally {
    parsing.value = false
  }
}

function removePark(idx) {
  parsed.value.parks.splice(idx, 1)
}

function removeScore(idx) {
  parsed.value.scores.splice(idx, 1)
}
```

```
async function confirmImport() {
  try {
    await ElMessageBox.confirm(
      `确认导入 ${parsed.value.parks.length} 个公园、${parsed.value.scores.length} 条评分、
      ${parsed.value.surveys.length} 条问卷?`,
      '确认导入'
    )
  } catch {
    return
  }

  importing.value = true
  try {
    const res = await api.post('/admin/upload/report/confirm', parsed.value)
    const imp = res.data.imported
    ElMessage.success(`导入完成: ${imp.parks} 个公园, ${imp.scores} 条评分, ${imp.surveys} 条问卷`)
    parsed.value = null
    uploadFile.value = null
  } catch (e) {
    ElMessage.error(e.response?.data?.detail || '导入失败')
  } finally {
    importing.value = false
  }
}

function reset() {
  parsed.value = null
  uploadFile.value = null
}
</script>

<style scoped>
.upload-section {
  text-align: center;
  padding: 40px 0;
}

.upload-box {
  width: 100%;
}

.upload-icon {
  font-size: 56px;
  color: #409eff;
}

.upload-text p {
  margin: 8px 0;
  font-size: 16px;
}
```

```
.upload-hint {
  color: #999;
  font-size: 13px;
}

.upload-info {
  margin-top: 20px;
  display: flex;
  align-items: center;
  justify-content: center;
}

.parse-progress {
  margin-top: 24px;
  text-align: center;
}

.progress-text {
  color: #999;
  margin-top: 8px;
}

.result-header {
  display: flex;
  justify-content: space-between;
  align-items: center;
}

.result-actions {
  margin-top: 24px;
  text-align: center;
}
</style>

// File: frontend/src/views/Comparison.vue
<template>
  <div class="page-shell comparison-page">
    <PageHero title="对比分析" description="支持最多 5 个遗址公园开展多维综合比较，从保护管理、科研价值与公共服务等真实指标中发现差异。" :image="portalHero" compact>
      <template #actions><el-button plain :disabled="!comparisonData.length" @click="exportNotice"><el-icon><Download /></el-icon> 导出说明</el-button></template>
    </PageHero>

    <section class="section-card compare-selector">
      <div class="compare-selector__control"><h2>选择对比公园</h2><el-select v-model="selectedParks"
multiple filterable collapse-tags :max-collapse-tags="2" placeholder="选择 2-5 个遗址公园" :multiple-
limit="5"><el-option v-for="park in
allParks" :key="park.id" :label="park.short_name||park.name" :value="park.id" /></el-select><el-button
```



```
type="primary" :disabled="selectedParks.length<2" :loading="loading" @click="loadComparison">开始对比</el-
button></div>

<div class="selected-parks"><span v-if="!selectedParkObjects.length" class="selected-parks__empty">
还没有选择对比对象</span><article v-for="park in
selectedParkObjects" :key="park.id"><div><strong>{{ park.short_name||park.name }}</strong><span>{{ park.province }} · {{ park.park_type }}
</span></div><button type="button" :aria-label="`移除${park.short_name||park.name}`"
@click="removePark(park.id)"><el-icon><Close /></el-icon></button></article><button v-
if="selectedParks.length<5" type="button" class="add-card" @click="focusSelect"><el-icon><Plus /></el-
icon><span>最多可选 5 个</span></button></div>

</section>

<StatusState v-if="error" type="error" title="对比数据加载失败" :description="error" action-label="重新
加载" @action="loadComparison" />

<StatusState v-else-if="!comparisonData.length" type="empty" title="请选择至少两个公园" description="对
比结果将使用当前平台的真实评分记录生成。" />

<template v-else>
<section class="compare-tabs"><el-tabs v-model="activeTab"><el-tab-pane label="维度对比"
name="dimension"/><el-tab-pane label="指标对比" name="indicator"/><el-tab-pane label="数据表格"
name="table"/></el-tabs></section>

<section v-show="activeTab==='dimension'" class="compare-main-grid"><div class="chart-card"><div
class="section-card__header"><div><h2>维度雷达对比</h2><p>各维度得分均值，满分 100。</p></div></div><div
ref="radarRef" class="chart-canvas tall"></div></div><div class="chart-card chart-card--wide"><div
class="section-card__header"><div><h2>核心维度对比</h2><p>同一维度下比较不同公园表现。</p></div></div><div
ref="barRef" class="chart-canvas tall"></div></div><aside class="section-card insight-panel"><div
class="section-card__header"><div><h2>对比洞察</h2><p>由当前真实数据直接计算。</p></div></div><div v-
for="insight in insights" :key="insight.title" class="insight-item"><span><el-icon><TrendCharts /></el-
icon></span><div><strong>{{ insight.title }}</strong><p>{{ insight.text }}</p></div></div></aside></sectio
n>

<section v-show="activeTab==='indicator'" class="chart-card"><div class="section-
card__header"><div><h2>全部指标对比</h2><p>指标较多时可横向滚动查看。</p></div></div><div ref="indicatorRef"
class="chart-canvas tall"></div></section>

<section v-show="activeTab==='table'" class="section-card compare-table"><div class="section-
card__header"><div><h2>维度得分总览</h2><p>表格中的数值均由已录入指标计算。</p></div></div><div class="table-
scroll"><table><thead><tr><th>遗址公园</th><th v-for="dim in dimensions" :key="dim">{{ dim }}</th><th>综合
均值</th></tr></thead><tbody><tr v-for="park in tableRows" :key="park.name"><td>{{ park.name }}</td><td v-
for="dim in dimensions" :key="dim">{{ park.dimensions[dim]??'-
'}}</td><td><strong>{{ park.average }}</strong></td></tr></tbody></table></div></section>

</template>

</div>
</template>

<script setup>
import { computed, nextTick, onBeforeUnmount, onMounted, ref, watch } from 'vue'
import { useRoute, useRouter } from 'vue-router'
import { ElMessage } from 'element-plus'
import * as echarts from 'echarts'
import api from '@/utils/api'
import PageHero from '@/components/portal/PageHero.vue'
import StatusState from '@/components/common/StatusState.vue'
```

```

import portalHero from '@assets/images/portal-hero.webp'
import parkCover from '@assets/images/park-cover-fallback.webp'
const route=useRoute();const router=useRouter();const allParks=ref([]);const selectedParks=ref([]);const
comparisonData=ref([]);const loading=ref(false);const error=ref('');const activeTab=ref('dimension');const
radarRef=ref();const barRef=ref();const indicatorRef=ref();const charts=[];const
chartColors=['#2563eb','#22c55e','#7c3aed','#f59e0b','#06b6d4'];let observer;let radarChart;let
barChart;let indicatorChart
const selectedParkObjects = computed(() => selectedParks.value.map(id => allParks.value.find(park =>
park.id === id)).filter(Boolean))
const dimensions = computed(() => [...new Set(comparisonData.value.flatMap(item =>
Object.values(item.scores || {}).map(score => score.dimension)))]))
const tableRows = computed(() => comparisonData.value.map(item => {
  const values = {}
  dimensions.value.forEach(dim => {
    const scores = Object.values(item.scores || {}).filter(score => score.dimension === dim).map(score =>
Number(score.score)).filter(Number.isFinite)
    values[dim] = scores.length ? Number((scores.reduce((sum, value) => sum + value, 0) /
scores.length).toFixed(1)) : null
  })
  const valid = Object.values(item.scores || {}).map(score => Number(score.score)).filter(Number.isFinite)
  return { name: item.park_name, dimensions: values, average: valid.length ? (valid.reduce((sum, value) =>
sum + value, 0) / valid.length).toFixed(1) : '-' }
}))
const insights = computed(() => {
  if (!tableRows.value.length) return []
  const best = [...tableRows.value].sort((a, b) => Number(b.average || 0) - Number(a.average || 0))[0]
  return [
    { title: `综合表现: ${best.name}`, text: `当前已录入指标综合均值为 ${best.average}, 在本组中最高。`,
    ...dimensions.value.slice(0, 3).map(dim => {
      const item = [...tableRows.value].filter(row => row.dimensions[dim] != null).sort((a, b) =>
b.dimensions[dim] - a.dimensions[dim])[0]
      return { title: `${dim}优势`, text: item ? `${item.name} 在该维度均值为 ${item.dimensions[dim]}` : '
该维度暂无可比较数据。' }
    })
  ],
})
})
onMounted(async()=>{const res=await
api.get('/parks',{params:{page_size:100},silent:true});allParks.value=res.data.items||[];const
initial=String(route.query.park_ids||'').split(',').map(Number).filter(Boolean).slice(0,5);selectedParks.v
alue=initial.filter(id=>allParks.value.some(p=>p.id===id));if(selectedParks.value.length>=2)loadComparison
());onBeforeUnmount(()=>{observer?.disconnect();charts.forEach(chart=>chart.dispose());}watch(activeTab,
async()=>{await nextTick();renderActiveCharts();resizeActiveCharts()})
async function
loadComparison(){if(selectedParks.value.length<2)return;loading.value=true;error.value='';try{const
res=await
api.get('/statistics/comparison',{params:{park_ids:selectedParks.value.join(',')},silent:true});comparison
Data.value=res.data||[];router.replace({query:{park_ids:selectedParks.value.join(',')});await
nextTick();renderCharts()}catch(e){error.value=e.response?.data?.detail||'请检查后端服务或登录状态。
'}finally{loading.value=false}}

```

```

function makeChart(el){if(!el)return null;const chart=echarts.init(el);charts.push(chart);observer||=new
ResizeObserver(resizeActiveCharts);observer.observe(el);return chart}
function
renderCharts(){charts.splice(0).forEach(chart=>chart.dispose());radarChart=null;barChart=null;indicatorCha
rt=null;renderActiveCharts()}
function renderActiveCharts(){
  if(activeTab.value==='dimension'&&!radarChart&&radarRef.value&&barRef.value){
    radarChart=makeChart(radarRef.value);barChart=makeChart(barRef.value)

    radarChart.setOption({color:chartColors,tooltip:{},legend:{top:0},radar:{indicator:dimensions.value.map(na
me=>({name,max:100})),splitArea:{areaStyle:{color:['#fff','#f8fbff']}},splitLine:{lineStyle:{color:'#dbe5f
2'}}},series:[{type:'radar',data:tableRows.value.map(row=>({name:row.name,value:dimensions.value.map(dim=>
row.dimensions[dim]||0),areaStyle:{opacity:.06}}))}]}))

    barChart.setOption({color:chartColors,tooltip:{trigger:'axis'},legend:{top:0},grid:{top:54,right:20,bottom:
36,left:42},xAxis:{type:'category',data:dimensions.value},yAxis:{type:'value',max:100,splitLine:{lineStyle:
{color:'#eef1f5'}}},series:tableRows.value.map(row=>({name:row.name,type:'bar',barMaxWidth:22,data:dimensi
ons.value.map(dim=>row.dimensions[dim]||0}))))}
  }
  if(activeTab.value==='indicator'&&!indicatorChart&&indicatorRef.value){
    const indicators=[...new Set(comparisonData.value.flatMap(item=>Object.keys(item.scores||{})))]
    indicatorChart=makeChart(indicatorRef.value)

    indicatorChart.setOption({color:chartColors,tooltip:{trigger:'axis'},legend:{top:0},grid:{top:54,right:20,
bottom:90,left:42},dataZoom:[{type:'inside'},{type:'slider',bottom:10}],xAxis:{type:'category',data:indica
tors,axisLabel:{rotate:40,fontSize:10}},yAxis:{type:'value',max:100},series:comparisonData.value.map(item=
>({name:item.park_name,type:'bar',data:indicators.map(code=>item.scores?.[code]?.score||0}))))}
  }
}
function
resizeActiveCharts(){if(activeTab.value==='dimension'){radarChart?.resize();barChart?.resize()}else
if(activeTab.value==='indicator'){indicatorChart?.resize()}}
function
removePark(id){selectedParks.value=selectedParks.value.filter(item=>item!==id);if(selectedParks.value.leng
th<2)comparisonData.value=[]}function focusSelect(){document.querySelector('.compare-selector .el-
select__wrapper')?.click()}function exportNotice(){ElMessage.info('当前后端尚未提供对比报告导出接口。')}
</script>

<style scoped lang="scss">
.compare-selector{display:grid;grid-template-columns:340px 1fr;overflow:hidden}.compare-
selector__control{padding:24px;display:grid;gap:14px;border-right:1px solid var(--border)}.compare-
selector__control h2{margin:0;font-size:18px}.selected-parks{padding:20px;display:flex;align-
items:center;gap:10px;overflow-x:auto}.selected-parks article{min-
width:230px;padding:9px;display:grid;grid-template-columns:54px 1fr auto;align-
items:center;gap:10px;border:1px solid var(--border);border-radius:12px}.selected-parks article
img{width:54px;height:54px;object-fit:cover;border-radius:9px}.selected-parks article div{min-
width:0;display:grid}.selected-parks strong{font-size:12px;white-space:nowrap;overflow:hidden;text-
overflow:ellipsis}.selected-parks article span{margin-top:5px;color:var(--text-tertiary);font-
size:10px}.selected-parks article button{border:0;background:none;color:var(--text-
tertiary);cursor:pointer}.selected-parks__empty{color:var(--text-tertiary);font-size:13px}.add-card{min-

```

```
width:112px;height:74px;display:grid;place-items:center;border:1px dashed var(--brand-100);border-
radius:12px;color:var(--brand-600);background:var(--brand-50);cursor:pointer}.add-card span{font-
size:10px}.compare-tabs{padding:0 16px;border:1px solid var(--border);border-
radius:14px;background:#fff}.compare-main-grid{display:grid;grid-template-columns:1fr
1.35fr .8fr;gap:16px}.insight-panel,.compare-table{padding:22px}.insight-
item{display:flex;gap:10px;padding:14px 0;border-bottom:1px solid var(--border)}.insight-
item>span{width:34px;height:34px;display:grid;place-items:center;flex:0 0 auto;border-
radius:10px;color:var(--brand-600);background:var(--brand-50)}.insight-item strong{font-
size:12px}.insight-item p{margin:5px 0 0;color:var(--text-secondary);font-size:11px;line-
height:1.6}.table-scroll{overflow-x:auto}.compare-table table{width:100%;border-collapse:collapse;font-
size:12px}.compare-table th,.compare-table td{padding:13px;border:1px solid var(--border);text-
align:center}.compare-table th{background:var(--surface-soft);color:var(--text-secondary)}.compare-table
td:first-child{text-align:left;font-weight:600}.compare-table td strong{color:var(--brand-600)}@media(max-
width:1150px){.compare-main-grid{grid-template-columns:1fr 1fr}.insight-panel{grid-column:1/-1}.compare-
selector{grid-template-columns:300px 1fr}}@media(max-width:760px){.compare-selector{grid-template-
columns:1fr}.compare-selector__control{border-right:0;border-bottom:1px solid var(--border)}.selected-
parks article{min-width:210px}.compare-main-grid{grid-template-columns:1fr}.insight-panel{grid-
column:auto}}
```

```
// File: frontend/src/views/Dashboard.vue
```

```
<template>
  <div class="home-page">
    <section class="home-hero">
      
      <div class="home-hero__content">
        <span class="home-hero__badge">国家考古遗址公园 · 数字研究门户</span>
        <h1>探索遗址公园<br /><em>数据 · 知识 · 智能</em></h1>
        <p>整合全国遗址公园数据、评价体系与知识资源，为文化遗产保护、科学研究与管理决策提供可信支持。</p>
        <div class="home-hero__actions">
          <el-button type="primary" size="large" @click="$router.push('/parks')">开始探索 <el-
            icon><ArrowRight /></el-icon></el-button>
          <el-button size="large" plain @click="$router.push('/about')">了解平台</el-button>
        </div>
      </div>
    </section>

    <section class="home-stats" aria-label="平台核心数据">
      <article v-for="item in statItems" :key="item.label" class="home-stat">
        <span :class="['home-stat__icon', item.tone]"><el-icon><component :is="item.icon" /></el-
          icon></span>

        <div><span>{{ item.label }}</span><strong>{{ item.value }}</strong><small>{{ item.meta }}</small></div>
      </article>
    </section>

    <section class="home-section">
      <div class="section-heading">
        <div><span class="eyebrow">RESEARCH TOOLKIT</span><h2>从数据到洞察，一站式展开遗址研究</h2></div>
      </div>
```

```
<div class="feature-grid">
  <router-link v-for="feature in features" :key="feature.path" :to="feature.path" class="feature-
card">
    <span :class="['feature-card__icon', feature.tone]"><el-icon><component :is="feature.icon"
/></el-icon></span>
    <div><h3>{{ feature.title }}</h3><p>{{ feature.description }}</p></div>
    <el-icon class="feature-card__arrow"><ArrowRight /></el-icon>
  </router-link>
</div>
</section>

<section class="home-content-grid">
  <div class="section-card research-panel">
    <div class="section-card__header">
      <div><span class="eyebrow">LATEST RESEARCH</span><h2>最新研究动态</h2><p>来自平台知识库的真实文献与
研究成果。</p></div>
      <router-link to="/library" class="section-heading__link">查看全部 <el-icon><ArrowRight /></el-
icon></router-link>
    </div>
    <StatusState v-if="researchLoading" type="loading" title="正在读取研究成果" />
    <StatusState v-else-if="!researchItems.length" type="empty" title="暂无公开研究成果" description="
知识库有新内容后会自动显示在这里。" />
    <div v-else class="research-list">
      <article v-for="item in researchItems" :key="item.id" class="research-item">
        <span class="research-item__type">{{ item.doc_type || '文献' }}</span>
        <div><h3>{{ item.title }}</h3><p>{{ authorText(item) }} · {{ item.year || '年份未知
' }}</p></div>
        <el-icon><ArrowRight /></el-icon>
      </article>
    </div>
  </div>

  <div class="section-card platform-panel">
    <div class="section-card__header"><div><span class="eyebrow">PLATFORM VALUE</span><h2>平台亮点
</h2></div></div>
    <div class="value-list">
      <div v-for="value in values" :key="value.title">
        <span :class="['value-list__icon', value.tone]"><el-icon><component :is="value.icon" /></el-
icon></span>
        <div><h3>{{ value.title }}</h3><p>{{ value.description }}</p></div>
      </div>
    </div>
  </div>
</section>

<section class="home-section">
  <div class="section-heading">
    <div><span class="eyebrow">HERITAGE PARKS</span><h2>遗址公园精选</h2><p>浏览平台当前收录的真实遗址公园
数据。</p></div>
```

```

      <router-link to="/parks" class="section-heading__link">全部公园 <el-icon><ArrowRight /></el-
icon></router-link>
    </div>
    <StatusState v-if="parksLoading" type="loading" title="正在读取公园数据" />
    <StatusState v-else-if="!parks.length" type="empty" title="暂无遗址公园数据" />
    <div v-else class="park-preview-grid">
      <router-link v-for="park in parks" :key="park.id" :to="`/parks/${park.id}`" class="park-preview-
card">
        
        <div class="park-preview-card__body">
          <span>第{{ park.batch || '-' }}批 · {{ park.park_type || '类型未录入' }}</span>
          <h3>{{ park.short_name || park.name }}</h3>
          <p><el-icon><Location /></el-icon>{{ park.province }} {{ park.city }}</p>
        </div>
      </router-link>
    </div>
  </section>

  <section class="home-callout">
    <div><span class="eyebrow">RESEARCH WITH EVIDENCE</span><h2>让数据赋能文化遗产保护，让知识驱动未来研究
</h2></div>
    <el-button type="primary" size="large" @click="$router.push('/research-data')">查看研究数据</el-
button>
  </section>
</div>
</template>

<script setup>
import { computed, onMounted, ref } from 'vue'
import {
  OfficeBuilding, Location, DataAnalysis, PieChart, MapLocation, TrendCharts,
  Share, ChatDotRound, Collection, Document, Search, Connection, CircleCheck,
} from '@element-plus/icons-vue'
import api from '@/utils/api'
import StatusState from '@/components/common/StatusState.vue'
import portalHero from '@/assets/images/portal-hero.webp'
import parkCover from '@/assets/images/park-cover-fallback.webp'

const stats = ref({})
const parks = ref([])
const researchItems = ref([])
const parksLoading = ref(true)
const researchLoading = ref(true)

const statItems = computed(() => [
  { label: '遗址公园数量', value: formatNumber(stats.value.total_parks), meta: `覆盖
${Object.keys(stats.value.by_province || {}).length} 个省级区域`, icon: OfficeBuilding, tone: 'blue' },
  { label: '遗址地点', value: formatNumber(stats.value.total_sites), meta: stats.value.total_sites ? '已录
入遗址点数据' : '当前暂无遗址点记录', icon: Location, tone: 'green' },

```

```

    { label: '评价指标', value: formatNumber(stats.value.total_indicators), meta: '多维度评价体系', icon:
DataAnalysis, tone: 'violet' },
    { label: '公园批次', value: `${Object.keys(stats.value.by_batch || {}).length}`, meta: '当前数据中的评定批
次', icon: PieChart, tone: 'orange' },
  ])

const features = [
  { title: '研究数据', description: '真实统计数据与多维可视化', path: '/research-data', icon: DataAnalysis,
tone: 'blue' },
  { title: '遗址公园', description: '浏览公园档案与评价指标', path: '/parks', icon: OfficeBuilding, tone:
'sand' },
  { title: '地图浏览', description: '探索空间分布与区域特征', path: '/map', icon: MapLocation, tone:
'green' },
  { title: '对比分析', description: '多公园综合指标横向比较', path: '/compare', icon: TrendCharts, tone:
'blue' },
  { title: '知识图谱', description: '发现实体关系与研究脉络', path: '/knowledge-graph', icon: Share, tone:
'violet' },
  { title: 'AI 助手', description: '以数据与知识辅助研究', path: '/assistant', icon: ChatDotRound, tone:
'cyan' },
  { title: '知识库', description: '文献、报告与知识资源', path: '/library', icon: Collection, tone:
'indigo' },
  { title: '文献计量', description: '洞察研究趋势与热点', path: '/bibliometrics', icon: Document, tone:
'orange' },
]

const values = [
  { title: '权威数据整合', description: '统一汇聚真实公园、指标与研究数据。', icon: CircleCheck, tone:
'blue' },
  { title: '多维研究工具', description: '从空间、评价到文献的完整分析链路。', icon: Search, tone: 'green' },
  { title: '知识智能联动', description: '知识图谱与 AI 助手连接研究线索。', icon: Connection, tone:
'violet' },
]

onMounted(() => {
  loadOverview()
  loadParks()
  loadResearch()
})

async function loadOverview() {
  try { stats.value = (await api.get('/statistics/overview', { silent: true })).data } catch { stats.value
= {} }
}

async function loadParks() {
  try { parks.value = (await api.get('/parks', { params: { page_size: 4 }, silent: true })).data.items ||
[] } finally { parksLoading.value = false }
}

async function loadResearch() {
  try { researchItems.value = (await api.get('/knowledge/list', { params: { page: 1, page_size: 4 },
silent: true })).data.items || [] } finally { researchLoading.value = false }
}

```

```
}  
function formatNumber(value) { return Number(value || 0).toLocaleString('zh-CN') }  
function authorText(item) { return (item.authors || []).slice(0, 2).map((author) =>  
author.name).filter(Boolean).join('、') || '作者未录入' }  
</script>  
  
<style scoped lang="scss">  
.home-page { display: grid; gap: 30px; }  
.home-hero { min-height: 470px; position: relative; overflow: hidden; border: 0; border-radius: 0;  
background: #f9fbff; box-shadow: none; }  
.home-hero__art { position: absolute; inset: 0; width: 100%; height: 100%; object-fit: cover; object-  
position: center; }  
.home-hero__content { width: 50%; min-height: 470px; padding: 68px 0 64px 64px; position: relative; z-  
index: 2; display: flex; flex-direction: column; justify-content: center; }  
.home-hero__badge { width: fit-content; margin-bottom: 18px; padding: 0 0 8px; border: 0; border-bottom:  
1px solid var(--brand-600); border-radius: 0; color: var(--brand-600); background: transparent; font-size:  
12px; font-weight: 650; }  
.home-hero h1 { margin: 0; font-size: clamp(42px, 4.4vw, 64px); line-height: 1.12; letter-spacing: -  
.045em; }  
.home-hero h1 em { color: var(--brand-600); font-style: normal; }  
.home-hero p { max-width: 600px; margin: 24px 0 0; color: #536078; font-size: 16px; line-height: 1.9; }  
.home-hero__actions { display: flex; gap: 12px; margin-top: 30px; }  
.home-stats { margin-top: -42px; padding: 0 32px; position: relative; z-index: 3; display: grid; grid-  
template-columns: repeat(4, minmax(0, 1fr)); background: rgba(255,255,255,.94); border-top: 1px solid  
var(--border); border-bottom: 1px solid var(--border); }  
.home-stat { min-height: 104px; padding: 20px 24px; display: flex; align-items: center; gap: 14px; border:  
0; border-right: 1px solid var(--border); background: transparent; box-shadow: none; }  
.home-stat:first-child { border-radius: 0; }  
.home-stat:last-child { border-right: 0; border-radius: 0; }  
.home-stat__icon, .feature-card__icon, .value-list__icon, .home-stat__icon.green, .home-  
stat__icon.violet, .home-stat__icon.orange, .feature-card__icon.green, .feature-  
card__icon.violet, .feature-card__icon.cyan, .feature-card__icon.orange, .feature-  
card__icon.sand, .feature-card__icon.indigo, .value-list__icon.green, .value-list__icon.violet { display:  
grid; place-items: center; flex: 0 0 auto; color: var(--brand-600); background: transparent; }  
.home-stat__icon { width: 34px; height: 34px; border-radius: 0; font-size: 22px; }  
.home-stat div { min-width: 0; display: grid; }.home-stat span { color: var(--text-secondary); font-size:  
12px; }.home-stat strong { margin-top: 2px; font-size: 25px; }.home-stat small { margin-top: 4px; color:  
var(--text-tertiary); white-space: nowrap; overflow: hidden; text-overflow: ellipsis; }  
.home-section { display: grid; gap: 18px; padding-top: 12px; }  
.feature-grid { display: grid; grid-template-columns: repeat(4, minmax(0, 1fr)); gap: 0; border-top: 1px  
solid var(--border); }  
.feature-card { min-height: 110px; padding: 24px 12px; display: grid; grid-template-columns: auto 1fr auto;  
align-items: start; gap: 14px; color: inherit; text-decoration: none; border: 0; border-bottom: 1px solid  
var(--border); border-radius: 0; background: transparent; box-shadow: none; transition: color .2s ease,  
background-color .2s ease; }  
.feature-card:hover { color: var(--brand-700); background: #f8fafc; }  
.feature-card__icon { width: 30px; height: 30px; border-radius: 0; font-size: 23px; }  
.feature-card h3 { margin: 3px 0 7px; font-size: 16px; }.feature-card p { color: var(--text-secondary);  
font-size: 12px; line-height: 1.6; }.feature-card__arrow { margin-top: 4px; color: var(--text-tertiary); }
```



```

.home-content-grid { display: grid; grid-template-columns: 1.35fr .8fr; gap: 0; padding-top: 24px; border-
top: 1px solid var(--border); }.research-panel,.platform-panel{padding:0 24px 0 0}.platform-
panel{padding:0 0 0 32px;border-left:1px solid var(--border)}.research-list{display:grid}.research-
item{min-height:76px;display:grid;grid-template-columns:auto 1fr auto;align-items:center;gap:14px;border-
bottom:1px solid var(--border)}.research-item:last-child{border-bottom:0}.research-item__type{padding:6px
9px;border-radius:4px;color:var(--brand-600);background:var(--brand-50);font-size:11px}.research-item
h3{margin:0;font-size:14px;line-height:1.5}.research-item p{margin:5px 0 0;color:var(--text-
tertiary);font-size:11px}.research-item>.el-icon{color:var(--text-tertiary)}
.value-list{display:grid;gap:20px}.value-list>div{display:flex;gap:13px}.value-
list__icon{width:30px;height:30px;border-radius:0;font-size:19px}.value-list h3{margin:1px 0 5px;font-
size:14px}.value-list p{color:var(--text-secondary);font-size:12px;line-height:1.6}
.park-preview-grid{display:grid;grid-template-columns:repeat(4,minmax(0,1fr));gap:24px}.park-preview-
card{overflow:hidden;color:inherit;text-decoration:none;border:0;border-
radius:0;background:transparent;box-shadow:none}.park-preview-card>img{width:100%;height:180px;object-
fit:cover;border-radius:4px;transition:transform .3s ease}.park-preview-
card:hover>img{transform:scale(1.025)}.park-preview-card__body{padding:14px 0 0}.park-preview-card__body
span{color:var(--brand-600);font-size:11px}.park-preview-card h3{margin:7px 0 10px;font-size:16px}.park-
preview-card p{display:flex;align-items:center;gap:5px;color:var(--text-secondary);font-size:12px}
.home-callout{min-height:160px;padding:36px 42px;display:flex;align-items:center;justify-content:space-
between;gap:24px;border-radius:var(--radius-xl);color:#fff;background:#162b57}.home-
callout .eyebrow{color:#8fb4ff}.home-callout h2{max-width:780px;margin:8px 0 0;font-size:25px;line-
height:1.4}
@media (max-width:1080px){.home-hero__content{width:62%;padding-left:44px}.home-stats{grid-template-
columns:repeat(2,1fr);margin-top:-40px}.home-stat:nth-child(2){border-right:0;border-radius:0}.home-
stat:nth-child(3){border-radius:0}.feature-grid,.park-preview-grid{grid-template-columns:repeat(2,1fr)}}
@media (max-width:760px){.home-page{gap:22px}.home-hero{min-height:520px}.home-hero__art{object-
position:64% center;opacity:.38}.home-hero__content{width:100%;min-height:520px;padding:44px 24px;justify-
content:flex-end}.home-hero h1{font-size:42px}.home-hero p{font-size:14px}.home-stats{margin-top:-
26px;padding:0 12px;display:flex;overflow-x:auto}.home-stat{min-width:235px;border-right:1px solid var(--
border);border-radius:0!important}.home-stat:last-child{border-right:0}.feature-grid,.park-preview-
grid,.home-content-grid{grid-template-columns:1fr}.feature-card{min-height:100px}.platform-
panel{padding:24px 0 0;border-left:0;border-top:1px solid var(--border)}.park-preview-
card>img{height:190px}.home-callout{align-items:flex-start;flex-direction:column;padding:28px}.home-
callout h2{font-size:21px}}
</style>

```

```
// File: frontend/src/views/MapView.vue
```

```

<template>
  <div class="map-page">
    <div :class="['map-workspace', { 'filter-collapsed': filterCollapsed, 'detail-
open': !!selectedPoint }]">
      <aside v-show="!filterCollapsed" class="map-side-panel">
        <div class="map-panel-tabs">
          <button class="active" type="button">公园筛选</button>
          <button type="button" @click="layerInfoVisible = true">图层管理</button>
        </div>
        <div class="map-filter-body">
          <div class="map-filter-group">
            <div><strong>公园类型</strong><button type="button" @click="filters.parkType = ''">清空
</button></div>

```

```

        <div class="chip-list">
            <button v-for="type in [' ', '城市型', '城郊型', '乡村型']" :key="type ||
'all'" :class="{ active: filters.parkType === type }" type="button" @click="filters.parkType =
type">{{ type || '全部' }}</button>
        </div>
    </div>
    <div class="map-filter-group">
        <div><strong>所属区域</strong><button type="button" @click="filters.province = ''">清空
</button></div>
        <el-select v-model="filters.province" clearable filterable placeholder="全部区域">
            <el-option v-for="province in provinces" :key="province" :label="province" :value="province"
/>

        </el-select>
    </div>
    <div class="map-filter-group">
        <div><strong>批次</strong><button type="button" @click="filters.batch = ''">清空</button></div>
        <div class="chip-list">
            <button :class="{ active: !filters.batch }" type="button" @click="filters.batch = ''">全部
</button>

            <button v-for="batch in batches" :key="batch" :class="{ active: Number(filters.batch) ===
Number(batch) }" type="button" @click="filters.batch = batch">第{{ batch }}批</button>
        </div>
    </div>
    <div class="map-filter-group map-filter-group--compact">
        <div><strong>最低综合分</strong><button type="button" @click="filters.minScore = 0">清空
</button></div>
        <el-slider v-model="filters.minScore" :min="0" :max="100" :show-input="false" />
    </div>
    <el-button type="primary" class="map-filter-submit" @click="applyFilters">筛选公园</el-button>
    <el-button class="map-filter-reset" @click="resetFilters">重置筛选</el-button>
</div>
<div class="map-park-list__section">
    <div class="map-park-list__label">公园列表 <span>{{ filteredPoints.length }} 个</span></div>
    <div class="map-park-list__body">
        <StatusState v-if="!filteredPoints.length" type="empty" title="暂无公园" description="调整筛选
条件后重试。" />
        <button
            v-for="point in filteredPoints"
            :key="point.id"
            type="button"
            :class="['map-park-card', { 'map-park-card--active': selectedPoint?.id === point.id }]"
            @click="selectPoint(point)"
        >
            
            <div class="map-park-card__overlay">
                <strong>{{ point.name }}</strong>
            </div>
            <span v-if="point.score != null" class="map-park-card__score">{{ point.score }}</span>
        </button>
    </div>
</div>

```

```
</div>
</aside>

<section class="map-canvas-shell">
  <button class="panel-collapse" type="button" :aria-label="filterCollapsed ? '展开筛选面板' : '收起筛选面板'" @click="toggleFilterPanel"><el-icon><DARrowLeft /></el-icon></button>
  <div ref="mapRef" class="map-canvas" aria-label="遗址公园专题地图"></div>
  <div class="map-utility-actions"><el-button size="small" plain @click="guideVisible=true"><el-icon><QuestionFilled /></el-icon> 使用指南</el-button><el-button size="small" plain @click="copyShareLink"><el-icon><Share /></el-icon> 分享</el-button></div>
  <StatusState v-if="mapStatus==='loading'" class="map-state" type="loading" title="正在加载地图服务" />
  <StatusState v-else-if="mapStatus==='missing-key'" class="map-state" type="error" title="地图服务暂不可用" description="请在前端环境变量中配置 VITE_AMAP_JS_KEY; 页面不会回退为假地图。" />
  <StatusState v-else-if="mapStatus==='error'" class="map-state" type="error" title="地图服务加载失败" :description="mapError" action-label="重新加载" @action="initMap" />
  <div v-if="mapStatus==='ready'" class="map-controls"><button type="button" title="回到全国" @click="resetView"><el-icon><Aim /></el-icon></button><button type="button" title="放大" @click="changeZoom(1)"><el-icon><Plus /></el-icon></button><button type="button" title="缩小" @click="changeZoom(-1)"><el-icon><Minus /></el-icon></button></div>
  <div class="map-legend"><strong>公园数量</strong><span><i></i></span> 当前筛选 {{ filteredPoints.length }} 个</span><small>点位经 WGS84 → GCJ-02 单一适配层转换</small></div>
  <button class="mobile-filter-button" type="button" @click="mobileFilterOpen=true"><el-icon><Filter /></el-icon>筛选</button>
  <div v-if="filteredPoints.length" class="map-mobile-park-strip">
    <button
      v-for="point in filteredPoints"
      :key="`mobile-${point.id}`"
      type="button"
      :class="['map-park-card', { 'map-park-card--active': selectedPoint?.id === point.id }]"
      @click="selectPoint(point)"
    >
      
      <div class="map-park-card__overlay">
        <strong>{{ point.name }}</strong>
        <small>{{ point.province }}</small>
      </div>
    </button>
  </div>
  <button v-if="!selectedPoint && filteredPoints.length" class="map-result-hint" type="button" @click="selectPoint(filteredPoints[0])">{{ filteredPoints.length }} 个结果 · 查看首个公园</button>
</section>

<aside v-if="selectedPoint" class="map-detail-panel">
  
  <div class="map-detail-panel__body"><div class="map-detail-panel__heading"><div><span>{{ selectedPoint.parkType || '类型未录入' }}</span><h2>{{ selectedPoint.name }}</h2></div><button type="button" aria-label="关闭详情" @click="selectPoint(null)"><el-icon><Close /></el-icon></button></div><p class="map-detail-panel__location"><el-icon><Location /></el-icon>{{ selectedPoint.province }}</p>
</div>
```

```

{{ selectedPoint.city }}<span class="map-detail-panel__batch">{{ selectedPoint.batch || '批次未录入'
' }}</span><h3>关键指标</h3><div class="map-detail-metrics"><div><span>综合评分
</span><strong>{{ selectedPoint.score??'-' }}</strong></div><div v-for="([name,value],index) in
Object.entries(selectedPoint.metrics)||{}).slice(0,3)" :key="name"><span>{{ index===0?'研究指标
':shortText(name) }}</span><strong>{{ value??'-' }}</strong></div></div><h3>公园简介</h3><p class="map-
detail-panel__summary">{{ selectedPoint.summary || '当前公园尚未录入简介。' }}</p><div class="map-detail-
panel__actions"><el-button type="primary" @click="$router.push(`/parks/${selectedPoint.id}`)">进入详情页
</el-button><el-button plain @click="$router.push({path: '/compare', query: {park_ids: selectedPoint.id}})">对
比分析</el-button></div></div>
</aside>
</div>

<el-drawer v-model="mobileFilterOpen" direction="btt" size="78%" title="筛选遗址公园" append-to-body>
<div class="mobile-filter-content">
<div class="map-filter-group"><strong>公园类型</strong><el-select v-model="filters.parkType"
clearable placeholder="全部类型"><el-option v-for="type in ['城市型','城郊型','乡村型
']" :key="type" :label="type" :value="type" /></el-select></div>
<div class="map-filter-group"><strong>所属区域</strong><el-select v-model="filters.province"
clearable filterable placeholder="全部区域"><el-option v-for="province in
provinces" :key="province" :label="province" :value="province" /></el-select></div>
<div class="map-filter-group"><strong>批次</strong><el-select v-model="filters.batch" clearable
placeholder="全部批次"><el-option v-for="batch in batches" :key="batch" :label="'第${batch}批
'" :value="batch" /></el-select></div>
<div class="map-filter-group"><strong>最低综合分</strong><el-slider v-
model="filters.minScore" :min="0" :max="100" show-input /></div>
<el-button type="primary" @click="applyMobileFilters">应用筛选</el-button>
<el-button @click="resetFilters(); mobileFilterOpen = false">重置筛选</el-button>
</div>
</el-drawer>

<el-dialog v-model="guideVisible" title="地图使用指南" width="min(560px,92vw)"><p class="guide-copy">使用
筛选面板缩小研究范围, 点击地图点位查看详情。筛选条件会同步到地址栏, 刷新或分享后仍可恢复。高德地图版权与审图信息保
留在地图画布中。</p></el-dialog>

<el-dialog v-model="layerInfoVisible" title="地图图层" width="min(520px,92vw)"><div class="layer-
list"><div><strong>浅色基础底图</strong><span>高德地图 JavaScript API 2.0</span></div><div><strong>遗址公园点
图层</strong><span>Loca 2.0 + MarkerCluster</span></div><div><strong>筛选交互层</strong><span>当前真实公园接
口数据</span></div></div></el-dialog>
</div>
</template>

<script setup>
import { computed, nextTick, onBeforeUnmount, onMounted, reactive, ref, watch } from 'vue'
import { useRoute, useRouter } from 'vue-router'
import { ElMessage } from 'element-plus'
import api from '@/utils/api'
import StatusState from '@/components/common/StatusState.vue'
import { parkToMapPoint } from '@/features/map/parkPointAdapter'
import { AMapProvider } from '@/features/map/provider/AMapProvider'
import parkCover from '@/assets/images/park-cover-fallback.webp'
const route=useRoute();const router=useRouter();const mapRef=ref();const mapStatus=ref('loading');const
mapError=ref('');const points=ref([]);const selectedPoint=ref(null);const filterCollapsed=ref(false);const

```

```

mobileFilterOpen=ref(false);const guideVisible=ref(false);const layerInfoVisible=ref(false);let
provider;let observer
const
filters=reactive({parkType:String(route.query.type|| ''),province:String(route.query.province|| ''),batch:ro
ute.query.batch?Number(route.query.batch):'',minScore:Number(route.query.minScore||0)})
const provinces=computed(()=>[...new Set(points.value.map(p=>p.province).filter(Boolean))]);const
batches=computed(()=>[...new
Set(points.value.map(p=>Number(String(p.batch|| '').replace(/D/g, ''))).filter(Boolean))].sort((a,b)=>a-b))
const
filteredPoints=computed(()=>points.value.filter(p=>(!filters.parkType||p.parkType===filters.parkType)&&(!f
ilters.province||p.province===filters.province)&&(!filters.batch||Number(String(p.batch|| '').replace(/D/g,
 ''))===Number(filters.batch))&&(!filters.minScore||Number(p.score||0)>=filters.minScore)))
onMounted(loadPoints);onBeforeUnmount(()=>{observer?.disconnect();provider?.destroy()})
async function loadPoints(){try{const res=await
api.get('/parks',{params:{page_size:100},silent:true});points.value=(res.data.items||[]).map(parkToMapPoin
t).filter(p=>Number.isFinite(p.longitude)&&Number.isFinite(p.latitude));await nextTick();await
initMap();const
selected=route.query.selected&&points.value.find(p=>String(p.id)===String(route.query.selected));if(select
ed)selectPoint(selected);else
if(points.value.length)selectPoint(points.value[0])}catch(e){mapStatus.value='error';mapError.value=e.resp
onse?.data?.detail||'无法读取公园点位数据。'}}
async function initMap(){provider?.destroy();mapError.value='';const
key=import.meta.env.VITE_AMAP_JS_KEY;if(!key){mapStatus.value='missing-
key';return}mapStatus.value='loading';try{provider=new
AMapProvider({key,securityCode:import.meta.env.VITE_AMAP_SECURITY_CODE,mapStyle:import.meta.env.VITE_AMAP_
MAP_STYLE});await
provider.mount(mapRef.value);provider.onPointClick(selectPoint);provider.setPoints(filteredPoints.value);o
bserver=new
ResizeObserver(()=>provider.resize());observer.observe(mapRef.value);mapStatus.value='ready'}catch(e){mapS
tatus.value=e.message==='MISSING_AMAP_KEY'? 'missing-key': 'error';mapError.value=e.message||'请检查地图服务
配置。'}}
function
applyFilters(){provider?.setPoints(filteredPoints.value);syncQuery();if(selectedPoint.value&&!filteredPoin
ts.value.some(p=>p.id===selectedPoint.value.id))selectedPoint.value=null}
function
resetFilters(){Object.assign(filters,{parkType:'',province:'',batch:'',minScore:0});applyFilters()}
function
syncQuery(){router.replace({query:{...(filters.province&&{province:filters.province}),...(filters.parkType
&&{type:filters.parkType}),...(filters.batch&&{batch:filters.batch}),...(filters.minScore&&{minScore:filte
rs.minScore}),...(selectedPoint.value&&{selected:selectedPoint.value.id})}})}
function
toggleFilterPanel(){filterCollapsed.value=!filterCollapsed.value;nextTick(()=>{provider?.resize();setTimeo
ut(()=>provider?.resize(),220)}})
watch(filterCollapsed,()=>nextTick(()=>{provider?.resize();setTimeout(()=>provider?.resize(),220)}))
function
selectPoint(point){selectedPoint.value=point;provider?.setSelectedPoint(point?.id||null);syncQuery();nextT
ick(()=>provider?.resize())}
function resetView(){provider?.setView({center:[104.2,35.8],zoom:4.6})}function changeZoom(delta){const
current=provider?.map?.getZoom?.()||5;provider?.map?.setZoom?.(current+delta)}function
applyMobileFilters(){mobileFilterOpen.value=false;applyFilters()}async function copyShareLink(){try{await

```

```

navigator.clipboard.writeText(window.location.href);ElMessage.success(' 分享链接已复制
'))}catch{ElMessage.warning(' 请从地址栏复制当前链接')}}function shortText(text){return String(text||'指标
').replace(/[(\).*$/,'').slice(0,7)}
</script>

<style scoped lang="scss">
.map-page{display:flex;flex-direction:column;height:100%;overflow:hidden;overscroll-behavior:none;touch-
action:pan-x pan-y}.map-workspace{flex:1;min-height:0;height:auto;position:relative;display:grid;grid-
template-columns:278px minmax(0,1fr) 320px;grid-template-rows:minmax(0,1fr);overflow:hidden;overscroll-
behavior:none;border:0;border-radius:0;background:#edf3fb;box-shadow:none;transition:grid-template-
columns .2s ease}.map-workspace.filter-collapsed{grid-template-columns:minmax(0,1fr) 320px}.map-
workspace.filter-collapsed:not(.detail-open){grid-template-columns:minmax(0,1fr)}.map-side-panel{grid-
column:1;grid-row:1;z-index:3;background:rgba(255,255,255,.97);min-width:0;min-
height:0;overflow:hidden;display:grid;grid-template-rows:auto auto minmax(0,1fr);border-right:1px solid
var(--border)}.map-canvas-shell{grid-column:2;grid-row:1;min-width:0;min-
height:0;position:relative;height:100%;overflow:hidden}.map-workspace.filter-collapsed .map-canvas-
shell{grid-column:1}.map-workspace.filter-collapsed.detail-open .map-canvas-shell{grid-column:1}.map-
detail-panel{grid-column:3;grid-row:1;z-index:3;background:rgba(255,255,255,.97);min-width:0;min-
height:0;overflow:hidden auto;border-left:1px solid var(--border)}.map-workspace.filter-collapsed .map-
detail-panel{grid-column:2}.map-panel-tabs{height:44px;display:flex;flex-shrink:0;border-bottom:1px solid
var(--border)}.map-panel-tabs button{flex:1;border:0;background:none;color:var(--text-
secondary);cursor:pointer;font-size:12px}.map-panel-tabs button.active{color:var(--brand-600);font-
weight:650;box-shadow:0 -2px 0 var(--brand-600) inset}.map-filter-body{flex-shrink:0;max-
height:42%;padding:12px 14px 10px;overflow-y:auto;border-bottom:1px solid var(--border)}.map-filter-
group{display:grid;gap:8px;padding:0 0 12px;margin-bottom:12px;border-bottom:1px solid var(--border)}.map-
filter-group:last-of-type{border-bottom:0;margin-bottom:10px;padding-bottom:0}.map-filter-group>div:first-
child{display:flex;align-items:center;justify-content:space-between}.map-filter-group strong{font-
size:12px}.map-filter-group>div:first-child button{border:0;background:none;color:var(--text-
tertiary);font-size:10px;cursor:pointer}.map-filter-group--compact{padding-bottom:8px;margin-
bottom:8px}.chip-list{display:flex;gap:5px;flex-wrap:wrap}.chip-list button{padding:5px 8px;border:1px
solid var(--border);border-radius:7px;color:var(--text-secondary);background:#fff;cursor:pointer;font-
size:10px}.chip-list button.active{border-color:var(--brand-600);color:#fff;background:var(--brand-
600)}.map-filter-submit,.map-filter-reset{width:100%;margin:0 0 6px}.map-filter-reset{margin-left:0}.map-
park-list__section{min-height:0;display:grid;grid-template-rows:auto minmax(0,1fr);overflow:hidden}.map-
park-list__label{padding:10px 14px 8px;font-size:12px;font-weight:650;flex-shrink:0}.map-park-list__label
span{margin-left:6px;color:var(--text-tertiary);font-size:11px;font-weight:400}.map-park-
list__body{padding:0 10px 10px;display:grid;grid-template-
columns:repeat(2,minmax(0,1fr));gap:8px;overflow-y:auto;align-content:start;overscroll-
behavior:contain}.map-park-card{position:relative;width:100%;height:72px;padding:0;border:2px solid
transparent;border-radius:10px;overflow:hidden;background:#edf2f8;cursor:pointer;text-
align:left;transition:border-color .2s ease}.map-park-card:hover{border-color:var(--brand-200)}.map-park-
card--active{border-color:var(--brand-600);box-shadow:0 0 0 1px var(--brand-100)}.map-park-card
img{width:100%;height:100%;object-fit:cover;display:block}.map-park-
card__overlay{position:absolute;inset:0;display:flex;align-items:flex-end;padding:8px;background:linear-
gradient(180deg,rgba(15,23,42,0) 10%,rgba(15,23,42,.82) 100%);color:#fff}.map-park-card__overlay
strong{font-size:11px;line-height:1.3;text-shadow:0 1px 3px rgba(0,0,0,.4);display:-webkit-box;-webkit-
line-clamp:2;-webkit-box-orient:vertical;overflow:hidden}.map-park-
card__score{position:absolute;top:5px;right:5px;min-width:26px;padding:2px 5px;border-
radius:6px;color:var(--brand-700);background:rgba(255,255,255,.92);font-size:10px;font-weight:700;text-
align:center}.panel-collapse{width:28px;height:40px;position:absolute;left:0;top:58px;z-index:6;border:1px

```

```

solid var(--border);border-left:0;border-radius:0 8px 8px 0;color:var(--text-
secondary);background:#fff;cursor:pointer;transition:transform .2s ease}.map-workspace:not(.filter-
collapsed) .panel-collapse{left:-1px}.map-workspace.filter-collapsed .panel-collapse{top:16px}.map-
workspace.filter-collapsed .panel-collapse .el-icon{transform:rotate(180deg)}.map-workspace.filter-
collapsed .map-controls{left:40px}.map-canvas{width:100%;height:100%;background:#edf3fb}.map-
state{position:absolute;inset:50% auto auto 50%;width:min(440px,calc(100% - 36px));min-
height:240px;transform:translate(-50%,-50%);z-index:4;box-shadow:var(--shadow-soft)}.map-
controls{position:absolute;left:16px;top:16px;display:grid;z-index:2;border:1px solid var(--
border);border-radius:11px;overflow:hidden;box-shadow:var(--shadow-soft)}.map-controls
button{width:38px;height:38px;border:0;border-bottom:1px solid var(--border);color:var(--text-
secondary);background:#fff;cursor:pointer}.map-controls button:last-child{border-bottom:0}.map-
legend{position:absolute;left:16px;bottom:16px;z-index:2;padding:13px 15px;display:grid;gap:6px;border:1px
solid var(--border);border-radius:11px;background:rgba(255,255,255,.94);box-shadow:var(--shadow-
soft);font-size:11px}.map-legend span{display:flex;align-items:center;gap:6px;color:var(--text-
secondary)}.map-legend i{width:9px;height:9px;border-radius:50%;background:var(--brand-600)}.map-legend
small{max-width:190px;color:var(--text-tertiary);line-height:1.5}.map-detail-
panel>img{width:100%;height:190px;object-fit:cover}.map-detail-panel__body{padding:20px}.map-detail-
panel__heading{display:flex;justify-content:space-between;gap:10px}.map-detail-panel__heading
span{color:var(--brand-600);font-size:10px}.map-detail-panel__heading h2{margin:6px 0 0;font-
size:21px;line-height:1.4}.map-detail-panel__heading button{width:32px;height:32px;border:1px solid var(--
border);border-radius:9px;background:#fff;cursor:pointer}.map-detail-panel__location{margin:9px
0;display:flex;align-items:center;gap:5px;color:var(--text-secondary);font-size:12px}.map-detail-
panel__batch{display:inline-block;padding:6px 9px;border-
radius:999px;color:#6d28d9;background:#f5f3ff;font-size:10px}.map-detail-panel h3{margin:20px 0 10px;font-
size:13px}.map-detail-metrics{display:grid;grid-template-columns:1fr 1fr;gap:8px}.map-detail-
metrics>div{padding:11px;border-radius:10px;background:var(--surface-soft)}.map-detail-metrics
span{display:block;color:var(--text-tertiary);font-size:9px}.map-detail-metrics
strong{display:block;margin-top:4px;font-size:16px}.map-detail-panel__summary{color:var(--text-
secondary);font-size:12px;line-height:1.75}.map-detail-panel__actions{display:grid;grid-template-
columns:1fr 1fr;gap:8px;margin-top:18px}.map-result-hint{position:absolute;right:18px;top:18px;z-
index:2;padding:10px 14px;border:1px solid var(--brand-100);border-radius:10px;color:var(--brand-
600);background:#fff;cursor:pointer;box-shadow:var(--shadow-soft)}.map-summary-bar{min-
height:96px;padding:16px 24px;display:grid;grid-template-columns:repeat(3,1fr) 1.6fr;align-
items:center;border:1px solid var(--border);border-radius:16px;background:#fff;box-shadow:var(--shadow-
soft)}.map-summary-bar>div{min-width:0;display:flex;align-items:center;gap:12px;padding:0 22px;border-
right:1px solid var(--border)}.map-summary-bar>div:first-child{padding-left:0}.map-summary-bar>div:last-
child{border:0}.map-summary-bar p{display:grid}.map-summary-bar small{color:var(--text-tertiary);font-
size:10px}.map-summary-bar strong{font-size:21px}.map-summary-distribution{display:flex!important;flex-
wrap:wrap!important}.map-summary-distribution span{padding:5px 8px;border-radius:999px;background:var(--
surface-soft);font-size:10px}.mobile-filter-button{display:none}.map-mobile-park-
strip{display:none}.guide-copy{color:var(--text-secondary);line-height:1.9}.layer-
list{display:grid;gap:10px}.layer-list>div{padding:14px;display:flex;justify-content:space-
between;gap:10px;border:1px solid var(--border);border-radius:11px}.layer-list span{color:var(--text-
secondary);font-size:12px}.mobile-filter-content{display:grid;gap:18px}.mobile-filter-content .map-filter-
group{margin:0}.mobile-filter-content>.el-button{width:100%}
@media(max-width:1200px){.map-workspace{grid-template-columns:260px minmax(0,1fr)}.map-workspace.filter-
collapsed{grid-template-columns:minmax(0,1fr)}.map-workspace.filter-collapsed.detail-open{grid-template-
columns:minmax(0,1fr) 320px}.map-detail-panel{position:absolute;right:0;top:0;bottom:0;width:320px;box-
shadow:-12px 0 30px rgba(31,45,61,.1)}.map-workspace.filter-collapsed .map-detail-panel{grid-
column:auto}.map-summary-bar{grid-template-columns:repeat(3,1fr)}.map-summary-

```

```

distribution{display:none!important}}@media(max-width:760px){.map-workspace{display:block;border-
radius:0}.map-side-panel{display:none}.map-canvas-shell{width:100%;height:100%}.mobile-filter-
button{display:flex;align-items:center;gap:6px;position:absolute;left:14px;top:14px;z-index:3;padding:10px
14px;border:1px solid var(--border);border-radius:10px;color:var(--brand-600);background:#fff;box-
shadow:var(--shadow-soft)}.map-controls{top:62px}.map-legend{bottom:14px}.map-detail-
panel{top:auto;left:0;width:100%;max-height:70%;border-radius:18px 18px 0 0}.map-detail-
panel>img{height:140px}.map-result-hint{right:12px;top:14px;max-width:54%;white-
space:nowrap;overflow:hidden;text-overflow:ellipsis}.map-state{min-height:210px}.map-detail-
panel__body{padding:16px}.map-mobile-park-strip{position:absolute;left:0;right:0;bottom:0;z-
index:3;display:flex;gap:8px;padding:10px 12px;overflow-x:auto;background:linear-
gradient(180deg,rgba(255,255,255,0) 0%,rgba(255,255,255,.96) 28%);scrollbar-width:none}.map-mobile-park-
strip::-webkit-scrollbar{display:none}.map-mobile-park-strip .map-park-card{flex:0 0 148px;height:92px}}
.map-utility-actions{position:absolute;right:16px;top:16px;z-index:3;display:flex;gap:8px}.map-utility-
actions .el-button{margin:0;background:rgba(255,255,255,.94)}.map-detail-panel__batch{border-
radius:4px;color:var(--brand-700);background:var(--brand-50)}.map-result-hint{top:58px;border-radius:6px}
@media(max-width:760px){.map-utility-actions{right:10px;top:10px}.map-detail-panel{border-radius:12px 12px
0 0}.map-result-hint{top:58px}}
</style>

```

```
// File: frontend/src/views/ParkDetail.vue
```

```
<template>
```

```
  <div class="page-shell park-detail-page">
```

```
    <StatusState v-if="loading" type="loading" title="正在加载公园详情" />
```

```
    <StatusState v-else-if="error" type="error" title="公园详情加载失败" :description="error" action-
label="返回列表" @action="$router.push('/parks')"/>
```

```
    <template v-else-if="park">
```

```
      <section class="detail-hero section-card">
```

```
        
```

```
        <div class="detail-hero__overlay"></div>
```

```
        <div class="detail-hero__content"><button type="button" class="back-button"
```

```
@click="$router.back()"><el-icon><ArrowLeft /></el-icon>返回</button><div class="detail-hero__tags"><span
v-if="park.batch">第{{ park.batch }}批</span><span>{{ park.park_type }}</span><span v-
if="park.world_heritage">世界遗产</span></div><h1>{{ park.name }}</h1><p><el-icon><Location /></el-
icon>{{ park.province }} {{ park.city }} {{ park.district || '' }}</p><div class="detail-
hero__actions"><el-button type="primary" @click="openMap">在地图中查看</el-button><el-button plain
@click="addToCompare">加入对比</el-button></div></div>
```

```
      </section>
```

```
      <div class="portal-stat-grid detail-stats">
```

```
        <div class="portal-stat-card"><span class="portal-stat-card__icon"><el-icon><DataAnalysis /></el-
icon></span><div><div class="portal-stat-card__label">综合均分</div><div class="portal-stat-
card__value">{{ averageScore }}</div><div class="portal-stat-card__meta">按已录入指标计算</div></div></div>
        <div class="portal-stat-card"><span class="portal-stat-card__icon green"><el-icon><MapLocation
/></el-icon></span><div><div class="portal-stat-card__label">总面积</div><div class="portal-stat-
card__value">{{ park.total_area ?? '-' }}</div><div class="portal-stat-card__meta">平方公里
</div></div></div>
```

```
        <div class="portal-stat-card"><span class="portal-stat-card__icon violet"><el-icon><Calendar
/></el-icon></span><div><div class="portal-stat-card__label">开园年份</div><div class="portal-stat-
card__value">{{ park.open_year || '-' }}</div><div class="portal-stat-card__meta">数据未录入时显示空值
</div></div></div>
```



```
<div class="portal-stat-card"><span class="portal-stat-card__icon orange"><el-icon><Location
/></el-icon></span><div><div class="portal-stat-card__label">遗址点</div><div class="portal-stat-
card__value">{{ sites.length }}</div><div class="portal-stat-card__meta">当前关联记录</div></div></div>
</div>

<section class="detail-grid">
  <article class="section-card detail-copy"><div class="section-card__header"><div><h2>公园简介
</h2><p>基础档案与保护信息。</p></div></div><p v-
if="park.description">{{ park.description }}</p><StatusState v-else type="empty" title="暂无公园简介"
description="当前数据库尚未录入相关文字介绍。" /><dl><div><dt>公园类型</dt><dd>{{ park.park_type || '-
' }}</dd></div><div><dt>评定批次</dt><dd>{{ park.batch ? `第${park.batch}批` : '-' }}</dd></div><div><dt>景区
等级</dt><dd>{{ park.aaa_level || '-' }}</dd></div><div><dt>世界遗产</dt><dd>{{ park.world_heritage ? '是
' : '否' }}</dd></div><div><dt>经纬度（数据库）</dt><dd>{{ coordinateText }}</dd></div><div><dt>核心保护区面
积</dt><dd>{{ park.core_area ?? '-' }}</dd></div></dl></article>

  <article class="chart-card"><div class="section-card__header"><div><h2>文化效能评估</h2><p>基于真实
评分记录的维度均值。</p></div></div><div v-if="scores.length" ref="radarRef" class="chart-canvas
tall"></div><StatusState v-else type="empty" title="暂无评价数据" /></article>
</section>

  <section class="section-card score-section"><div class="section-card__header"><div><h2>评价指标明细
</h2><p>展示指标编号、维度、得分等级与现有依据。</p></div></div><div class="table-scroll"><el-table v-
if="scores.length" :data="scores" stripe><el-table-column prop="code" label="编号" width="90"/><el-table-
column prop="name" label="指标名称" min-width="190"/><el-table-column prop="dimension" label="维度"
width="140"/><el-table-column prop="score" label="得分" width="90"><template #default="{row}"><strong
class="score-value">{{ row.score ?? '-' }}</strong></template></el-table-column><el-table-column
prop="grade" label="等级" width="100"><template #default="{row}"><el-
tag :type="gradeType(row.grade)">{{ row.grade || '-' }}</el-tag></template></el-table-column><el-table-
column prop="evidence" label="评分依据" min-width="280"><template #default="{row}">{{ row.evidence || '暂未
录入' }}</template></el-table-column></el-table><StatusState v-else type="empty" title="暂无评分明细"
/></div></section>

  <section class="detail-bottom-grid">
    <article class="section-card"><div class="section-card__header"><div><h2>遗址构成</h2><p>当前关联遗
址点。</p></div></div><div v-if="sites.length" class="site-list"><div v-for="site in
sites" :key="site.id"><strong>{{ site.site_name }}</strong><span>{{ site.site_type || '类型未录入
' }} · {{ site.period || '时期未录入' }}</span><p>{{ site.description || '暂无简介
' }}</p></div></div><StatusState v-else type="empty" title="暂无遗址点记录" description="当前数据库没有关联到
该公园的遗址点。" /></article>

    <article class="section-card location-card"><div class="section-card__header"><div><h2>地图位置
</h2><p>坐标由地图适配层统一转换并渲染。</p></div></div><div class="coordinate-panel"><span><el-
icon><MapLocation /></el-icon></span><div><strong>{{ park.province }}
{{ park.city }}</strong><p>{{ coordinateText }}</p></div></div><el-button type="primary"
plain :disabled="!park.longitude || !park.latitude" @click="openMap">打开专题地图</el-button></article>
  </section>
</template>
</div>
</template>

<script setup>
import { computed, nextTick, onBeforeUnmount, onMounted, ref } from 'vue'
```

```

import { useRoute, useRouter } from 'vue-router'
import * as echarts from 'echarts'
import api from '@/utils/api'
import StatusState from '@/components/common/StatusState.vue'
import parkCover from '@/assets/images/park-cover-fallback.webp'
const route=useRoute();const router=useRouter();const park=ref(null);const scores=ref([]);const
sites=ref([]);const loading=ref(true);const error=ref('');const radarRef=ref();let chart;let observer
const averageScore=computed(()=>{const
values=scores.value.map(s=>Number(s.score)).filter(Number.isFinite);return
values.length?(values.reduce((a,b)=>a+b,0)/values.length).toFixed(1):'-'})
const
coordinateText=computed(()=>park.value?.longitude&&park.value?.latitude?`${park.value.longitude.toFixed(4)}
, ${park.value.latitude.toFixed(4)} (WGS84)`:'暂无坐标')
onMounted(loadDetail);onBeforeUnmount(()=>{observer?.disconnect();chart?.dispose()})
async function loadDetail(){loading.value=true;try{const [parkRes,scoreRes,siteRes]=await
Promise.all([api.get(`/parks/${route.params.id}`, {silent:true}),api.get(`/parks/${route.params.id}/scores`,
{silent:true}),api.get(`/parks/${route.params.id}/sites`, {silent:true})]);park.value=parkRes.data;scores.v
alue=scoreRes.data||[];sites.value=siteRes.data||[];await
nextTick();renderRadar()}catch(e){error.value=e.response?.data?.detail||'请检查公园编号或后端服务。
'}finally{loading.value=false}}
function renderRadar(){if(!radarRef.value||!scores.value.length)return;const dims=[...new
Set(scores.value.map(s=>s.dimension))];const values=dims.map(dim=>{const
items=scores.value.filter(s=>s.dimension===dim).map(s=>Number(s.score)).filter(Number.isFinite);return
items.length?Math.round(items.reduce((a,b)=>a+b,0)/items.length):0});chart=echarts.init(radarRef.value);ch
art.setOption({color:['#2563eb'],tooltip:{},radar:{indicator:dims.map(name=>({name,max:100})),splitArea:{a
reaStyle:{color:['#fff','#f8fbff']},splitLine:{lineStyle:{color:'#dbe5f2'}},axisLine:{lineStyle:{color:'#
dbe5f2'}}},series:[{type:'radar',data:[{name:park.value.short_name|park.value.name,value:values,areaStyle:
{color:'rgba(37,99,235,.16)',lineStyle:{width:2}}}],observer=new
ResizeObserver(()=>chart?.resize());observer.observe(radarRef.value)}
function openMap(){router.push({path:'/map',query:{selected:String(park.value.id)}})}function
addToCompare(){router.push({path:'/compare',query:{park_ids:String(park.value.id)}})}function
gradeType(grade){return {'好':'success','较好':'primary','一般':'warning','较差':'danger','差
':'danger'}[grade]||'info'}
</script>

```

```

<style scoped lang="scss">
.detail-hero{min-height:410px;position:relative;overflow:hidden;color:#fff}.detail-
hero>img{position:absolute;inset:0;width:100%;height:100%;object-fit:cover}.detail-
hero__overlay{position:absolute;inset:0;background:rgba(12,30,56,.62)}.detail-hero__content{min-
height:410px;padding:38px;position:relative;z-index:1;display:flex;flex-direction:column;justify-
content:flex-end;align-items:flex-start}.back-
button{position:absolute;left:34px;top:28px;display:flex;align-
items:center;gap:5px;border:0;background:none;color:#fff;cursor:pointer}.detail-
hero__tags{display:flex;gap:8px}.detail-hero__tags span{padding:6px 10px;border:1px solid
rgba(255,255,255,.32);border-radius:999px;background:rgba(255,255,255,.12);font-size:11px}.detail-hero
h1{max-width:850px;margin:14px 0 10px;font-size:clamp(32px,4vw,48px);line-height:1.18}.detail-
hero__content>p{display:flex;align-items:center;gap:6px;opacity:.86}.detail-
hero__actions{display:flex;gap:10px;margin-top:20px}.detail-grid{display:grid;grid-template-columns:.85fr
1.15fr;gap:16px}.detail-copy,.score-section,.detail-bottom-grid>.section-card{padding:24px}.detail-
copy>p{margin:0 0 20px;color:var(--text-secondary);font-size:14px;line-height:1.9}.detail-copy

```

```
dl{display:grid;grid-template-columns:1fr 1fr;border-top:1px solid var(--border)}}.detail-copy
dl>div{padding:14px 0;border-bottom:1px solid var(--border)}.detail-copy dt{color:var(--text-
tertiary);font-size:11px}.detail-copy dd{margin:5px 0 0;font-size:13px;font-weight:600}.table-
scroll{overflow-x:auto}.score-value{color:var(--brand-600)}.detail-bottom-grid{display:grid;grid-template-
columns:1.2fr .8fr;gap:16px}.site-list{display:grid;gap:10px}.site-list>div{padding:14px;border:1px solid
var(--border);border-radius:12px}.site-list span{margin-left:10px;color:var(--text-tertiary);font-
size:11px}.site-list p{margin:7px 0 0;color:var(--text-secondary);font-size:12px}.coordinate-
panel{display:flex;align-items:center;gap:14px;margin-bottom:20px;padding:20px;border-
radius:14px;background:var(--brand-50)}.coordinate-panel>span{width:44px;height:44px;display:grid;place-
items:center;border-radius:13px;color:#fff;background:var(--brand-600);font-size:21px}.coordinate-panel
p{margin:6px 0 0;color:var(--text-secondary);font-size:12px}@media(max-width:900px){.detail-grid,.detail-
bottom-grid{grid-template-columns:1fr}}@media(max-width:640px){.detail-hero,.detail-hero__content{min-
height:470px}.detail-hero__content{padding:24px}.back-button{left:22px;top:22px}.detail-copy dl{grid-
template-columns:1fr}.detail-hero__actions{width:100%}.detail-hero__actions>{*{flex:1}}
</style>
```

```
// File: frontend/src/views/ResearchData.vue
```

```
<template>
```

```
  <div class="page-shell research-data-page">
```

```
    <PageHero title="研究数据" accent="数据总览" description="全面汇聚遗址公园真实数据，洞察类型、区域与评价维
度，为保护研究和管理决策提供清晰证据。" :image="portalHero" compact />
```

```
    <div class="portal-stat-grid">
```

```
      <div v-for="item in statItems" :key="item.label" class="portal-stat-card">
```

```
        <span :class="['portal-stat-card__icon', item.tone]"><el-icon><component :is="item.icon" /></el-
icon></span>
```

```
        <div class="portal-stat-card__copy"><div class="portal-stat-
card__label">{{ item.label }}</div><div class="portal-stat-card__value">{{ item.value }}</div><div
class="portal-stat-card__meta">{{ item.meta }}</div></div>
```

```
      </div>
```

```
    </div>
```

```
    <StatusState v-if="loading" type="loading" title="正在加载研究数据" />
```

```
    <StatusState v-else-if="error" type="error" title="研究数据加载失败" :description="error" action-
label="重新加载" @action="loadData" />
```

```
    <template v-else>
```

```
      <div class="research-chart-grid">
```

```
        <div class="chart-card chart-card--wide"><div class="section-card__header"><div><h2>评价维度概览
</h2><p>不同遗址公园类型的真实平均得分。</p></div></div><div ref="dimensionRef" class="chart-
canvas"></div></div>
```

```
        <div class="chart-card"><div class="section-card__header"><div><h2>公园类型分布</h2><p>当前数据库中的
类型构成。</p></div></div><div ref="typeRef" class="chart-canvas"></div></div>
```

```
        <div class="chart-card"><div class="section-card__header"><div><h2>省级区域分布</h2><p>按收录公园数
量排序。</p></div></div><div ref="provinceRef" class="chart-canvas"></div></div>
```

```
      </div>
```

```
    <div class="research-lower-grid">
```

```
      <div class="section-card insight-card">
```

```
        <div class="section-card__header"><div><h2>数据洞察</h2><p>基于当前真实数据的直接观察。</p></div></div>
```

```
    </div></div>
```

```
<ul>
  <li>当前共收录 <strong>{{ stats.total_parks || 0 }}</strong> 个遗址公园，覆盖
<strong>{{ provinceCount }}</strong> 个省级区域。</li>
  <li>公园数量最多的类型为 <strong>{{ topType.name }}</strong>，共 {{ topType.value }} 个。</li>
  <li>评价体系包含 <strong>{{ stats.total_indicators || 0 }}</strong> 项指标，已形成
{{ dimensions.length }} 个可统计维度。</li>
  <li v-if="!stats.total_sites">当前数据库暂无遗址点记录，相关模块会显示明确空状态。</li>
</ul>
<el-button type="primary" plain @click="$router.push('/parks')">浏览公园档案</el-button>
</div>
<div class="section-card quick-card">
  <div class="section-card__header"><div><h2>快速入口</h2><p>从数据概览继续深入研究。</p></div></div>
  <div class="quick-grid">
    <router-link v-for="item in quickLinks" :key="item.path" :to="item.path"><el-
icon><component :is="item.icon" /></el-
icon><span><strong>{{ item.title }}</strong><small>{{ item.description }}</small></span><el-
icon><ArrowRight /></el-icon></router-link>
  </div>
</div>
</div>
<div class="section-card update-card">
  <div class="section-card__header"><div><h2>数据更新说明</h2><p>当前接口未提供独立的数据版本和更新时间字
段。</p></div></div>
  <StatusState type="empty" title="暂无可展示的更新日志" description="为保持数据可信，本页面不会编造更新
时间或增长趋势；后端提供真实元数据后可在此展示。" />
</div>
</template>
</div>
</template>

<script setup>
import { computed, nextTick, onBeforeUnmount, onMounted, ref } from 'vue'
import * as echarts from 'echarts'
import { OfficeBuilding, Location, DataAnalysis, PieChart, MapLocation, TrendCharts, Collection } from
'@element-plus/icons-vue'
import api from '@/utils/api'
import PageHero from '@/components/portal/PageHero.vue'
import StatusState from '@/components/common/StatusState.vue'
import portalHero from '@/assets/images/portal-hero.webp'

const stats = ref({})
const dimensions = ref([])
const loading = ref(true)
const error = ref('')
const dimensionRef = ref()
const typeRef = ref()
const provinceRef = ref()
const charts = []
let observer
```

```
const provinceCount = computed(() => Object.keys(stats.value.by_province || {}).length)
const topType = computed(() => Object.entries(stats.value.by_type || {}).map(([name, value]) => ({ name,
value }))).sort((a, b) => b.value - a.value)[0] || { name: '暂无', value: 0 })
const statItems = computed(() => [
  { label: '遗址公园总数', value: Number(stats.value.total_parks || 0).toLocaleString('zh-CN'), meta: `覆盖
${provinceCount.value} 个省级区域`, icon: OfficeBuilding, tone: '' },
  { label: '遗址地点', value: Number(stats.value.total_sites || 0).toLocaleString('zh-CN'), meta:
stats.value.total_sites ? '已录入地点信息' : '当前暂无记录', icon: Location, tone: 'green' },
  { label: '评价指标', value: Number(stats.value.total_indicators || 0).toLocaleString('zh-CN'), meta: '真
实评价体系指标', icon: DataAnalysis, tone: 'violet' },
  { label: '评定批次', value: Object.keys(stats.value.by_batch || {}).length, meta: '当前数据中的批次', icon:
PieChart, tone: 'orange' },
])
const quickLinks = [
  { title: '遗址公园', description: '浏览真实公园档案', path: '/parks', icon: OfficeBuilding },
  { title: '地图浏览', description: '查看空间分布', path: '/map', icon: MapLocation },
  { title: '对比分析', description: '比较评价指标', path: '/compare', icon: TrendCharts },
  { title: '知识库', description: '检索研究文献', path: '/library', icon: Collection },
]

onMounted(loadData)
onBeforeUnmount(() => { observer?.disconnect(); charts.forEach((chart) => chart.dispose()) })

async function loadData() {
  loading.value = true
  error.value = ''
  try {
    const [overview, dimension] = await Promise.all([
      api.get('/statistics/overview', { silent: true }),
      api.get('/statistics/dimension', { silent: true }),
    ])
    stats.value = overview.data || {}
    dimensions.value = dimension.data || []
    loading.value = false
    await nextTick()
    renderCharts()
  } catch (e) {
    error.value = e.response?.data?.detail || '请检查后端服务后重试。'
    loading.value = false
  }
}

function createChart(el) {
  const chart = echarts.init(el)
  charts.push(chart)
  observer ||= new ResizeObserver(() => charts.forEach((item) => item.resize()))
  observer.observe(el)
  return chart
}
```

```

function renderCharts() {
  charts.splice(0).forEach((chart) => chart.dispose())
  const palette = ['#2563eb', '#22c55e', '#7c3aed', '#f59e0b', '#06b6d4']
  const dimNames = [...new Set(dimensions.value.map((item) => item.dimension))]
  const typeNames = [...new Set(dimensions.value.map((item) => item.park_type))]
  createChart(dimensionRef.value).setOption({ color: palette, tooltip: { trigger: 'axis' }, legend: { top:
0 }, grid: { top: 52, right: 20, bottom: 34, left: 46 }, xAxis: { type: 'category', data: dimNames,
axisLabel: { color: '#667085' }, axisLine: { lineStyle: { color: '#e7eaf0' } } }, yAxis: { type: 'value',
max: 100, splitLine: { lineStyle: { color: '#eef1f5' } } }, series: typeNames.map((type) => ({ name: type,
type: 'bar', barMaxWidth: 24, data: dimNames.map((dim) => dimensions.value.find((item) => item.park_type
=== type && item.dimension === dim)?.avg_score || 0) })))
  const typeData = Object.entries(stats.value.by_type || {}).map(([name, value]) => ({ name, value }))
  createChart(typeRef.value).setOption({ color: palette, tooltip: { trigger: 'item' }, legend: { bottom:
0 }, series: [{ type: 'pie', radius: ['48%', '70%'], center: ['50%', '44%'], label: { formatter:
'{b}\n{d}%' }, data: typeData ] })
  const provinceData = Object.entries(stats.value.by_province || {}).map(([name, value]) => ({ name,
value })).sort((a, b) => a.value - b.value)
  createChart(provinceRef.value).setOption({ color: ['#3b82f6'], tooltip: { trigger: 'axis' }, grid: { top:
12, right: 24, bottom: 24, left: 72 }, xAxis: { type: 'value', splitLine: { lineStyle: { color:
'#eef1f5' } } }, yAxis: { type: 'category', data: provinceData.map((item) => item.name), axisLine: { show:
false }, axisTick: { show: false } }, series: [{ type: 'bar', data: provinceData.map((item) => item.value),
barMaxWidth: 18, itemStyle: { borderRadius: [0, 7, 7, 0] } } ] })
}
</script>

<style scoped lang="scss">
.research-chart-grid { display: grid; grid-template-columns: 1.35fr .8fr; gap: 16px; }.chart-card--
wide{grid-row:span 2}.research-lower-grid{display:grid;grid-template-columns:1fr 1.2fr;gap:16px}.insight-
card,.quick-card,.update-card{padding:24px}.insight-card ul{display:grid;gap:13px;margin:0 0 20px;padding-
left:20px;color:var(--text-secondary);font-size:13px;line-height:1.7}.insight-card strong{color:var(--
brand-600)}.quick-grid{display:grid;grid-template-columns:1fr 1fr;gap:10px}.quick-grid a{min-
height:76px;padding:14px;display:grid;grid-template-columns:auto 1fr auto;align-
items:center;gap:10px;color:inherit;text-decoration:none;border:1px solid var(--border);border-
radius:12px}.quick-grid>a>.el-icon:first-child{width:36px;height:36px;display:grid;place-
items:center;color:var(--brand-600);background:var(--brand-50);border-radius:11px}.quick-grid
span{display:grid}.quick-grid strong{font-size:13px}.quick-grid small{margin-top:4px;color:var(--text-
tertiary)}.quick-grid>a>.el-icon:last-child{color:var(--text-tertiary)}
@media(max-width:980px){.research-chart-grid,.research-lower-grid{grid-template-columns:1fr}.chart-card--
wide{grid-row:auto}}@media(max-width:640px){.quick-grid{grid-template-columns:1fr}}
</style>

```